Архитектурное решение "Системы управления системами проектирования"

Содержание

1. Общее описание системы . . . 1
1.1. Описание системы . . . 1
1.1.1. Ограничения . . . 1
1.1.2. Условия . . . 1
2. Требования . . . 2
2.1. Функциональные требования . . . 2
2.2. Нефункциональные требования . . . 3
2.2.1. Производительность . . . 3
2.2.2. Надежность и доступность . . . 3
2.2.3. Масштабирование и развитие . . . 3
2.2.4. Безопасность . . . 4
2.2.5. Эргономика и удобство использования . . . 4
2.2.6. Сопровождаемость . . . 4
2.2.7. Интеграции . . . 5
2.2.8. Документирование . . . 5
2.3. Риски . . . 5
2.3.1. Инфраструктурные риски . . . 5
2.3.2. Бизнес-риски . . . 6
2.3.3. Риски интеграций . . . 6
2.3.4. Операционные риски . . . 7
2.3.5. Риски безопасности . . . 7
2.3.6. Риски непрерывности . . . 8
3. Концептуальная архитектура . . . 9
3.1. Схема концептуальной архитектуры . . . 9
3.2. Описание концептуальной архитектуры . . . 9
3.3. Пользователи решения . . . 11
3.3.1. Внешние пользователи . . . 11
3.3.2. Внутренние пользователи . . . 11
3.3.3. Внешние системы . . . 11
3.4. Внешние интеграции . . . 11
3.4.1. Интеграция с платёжными агрегаторами . . . 11
3.4.2. Интеграция с транспортной компанией . . . 12
3.5. Потоки данных и высокоуровневое взаимодействие . . . 13
3.5.1. Информационные потоки . . . 13
3.6. Потоки данных и высокоуровневое взаимодействие . . . 13
3.6.1. Потоки данных . . . 13
3.6.2. Схема взаимодействия . . . 14
3.7. Модель данных . . . 15

3.8. Расчеты показателей решения . . . 16
3.8.1. Исходные данные . . . 16
3.8.2. Распределение пользователей . . . 17
3.8.3. Пользовательская активность . . . 17
3.8.4. Суточная нагрузка . . . 17
3.8.5. Пиковая и всплесковая нагрузка . . . 17
3.8.6. Публикация объявлений . . . 18
3.8.7. Медиа-данные . . . 18
3.8.8. Структурированные данные . . . 18
3.8.9. Сетевой трафик . . . 18
3.8.10. Вывод . . . 19
3.9. Схема архитектуры решения . . . 19
3.10. Observability . . . 20
3.10.1. Показатели . . . 20
3.10.2. Средства . . . 21
3.10.3. Инструменты диагностики . . . 21
3.11. Реагирование на инциденты . . . 21
3.11.1. Роли и ответственность . . . 21
3.11.2. Мониторинг и обнаружение . . . 22
3.11.3. Классификация инцидентов . . . 22
3.11.4. Сценарии действий (Runbooks) . . . 23
3.11.5. Коммуникация . . . 24
3.11.6. Пост-инцидентный анализ . . . 24
3.12. Схема взаимодействия . . . 24
3.12.1. Принципы взаимодействия . . . 24
3.12.2. Синхронные взаимодействия (прямые вызовы сервис→сервис) . . . 25
3.12.3. Асинхронные взаимодействия (через Kafka) . . . 25
3.12.4. Внешние взаимодействия (Webhook) . . . 27
3.12.5. Схемы процессов . . . 27
3.13. Форматы взаимодействия . . . 30
3.13.1. DNS / GeoDNS . . . 30
3.13.2. HTTP/REST (JSON) . . . 30
3.13.3. gRPC (Protobuf) . . . 33
3.13.4. HTTP Webhook (JSON) . . . 35
3.13.5. Kafka / Event bus . . . 36
3.13.6. Redis / Cache (RESP) . . . 37
3.13.7. S3 / CDN . . . 38
3.14. Система хранения данных . . . 38
3.14.1. Общая архитектура . . . 38
3.14.2. Хранилища по сервисам . . . 38
3.14.3. Репликация и шардирование . . . 40

3.14.4. CQRS и Event-driven . . . 40
3.14.5. Итоговая модель хранения данных . . . 40
3.15. Схемы данных . . . 41
3.15.1. Пользователи и роли . . . 42
3.15.2. Объявления и сообщения . . . 42
3.15.3. Заказы и платежи . . . 45
3.15.4. Доставка . . . 48
3.15.5. Поддержка . . . 48
4. Безопасность . . . 50
4.1. Схема защиты и управления доступом . . . 50
4.1.1. Общие принципы . . . 50
4.1.2. Ролевая модель (RBAC) . . . 50
4.1.3. Аутентификация и сессии . . . 51
4.1.4. Управление секретами и ключами . . . 52
4.1.5. Шифрование . . . 53
4.1.6. Реализация контроля доступа . . . 53
4.1.7. Защита от атак . . . 53
4.1.8. Путь запроса с проверкой полномочий . . . 53
4.2. Аудит безопасности . . . 54
4.2.1. Цели . . . 55
4.2.2. Объекты аудита . . . 56
4.2.3. Источники данных . . . 56
4.2.4. Роли . . . 56
4.2.5. Методы контроля . . . 56
4.2.6. Управление инцидентами . . . 57
4.2.7. Административная проверка . . . 57
4.2.8. Контроль соответствия ролям . . . 57
5. Доставка решения . . . 58
5.1. Схема развертывания . . . 58
5.1.1. География и ЦОД . . . 58
5.1.2. Сервисы и распределение по ЦОД . . . 59
5.1.3. Принципы . . . 62
5.2. CI/CD процесс . . . 63
5.2.1. Инструменты . . . 63
5.2.2. Уровни критичности сервисов . . . 63
5.2.3. Архитектура пайплайнов . . . 63
5.2.4. Этапы CI/CD . . . 64
5.2.5. Мониторинг после деплоя и rollback . . . 66
5.2.6. Планы тестирования . . . 68
5.2.7. Объекты тестирования . . . 68
5.2.8. Архитектурные особенности . . . 69

5.2.9. Основные группы тестирования 70
5.2.10. Критические зоны системы 73
5.2.11. Общий критерий успешности тестирования 73

Глава 1. Общее описание системы

1.1. Описание системы

Рассматриваемая система является электронной торговой площадкой, предоставляющей сервис публикации объявлений о продаже товаров и предоставлении услуг с доставкой товаров и проведением оплаты.

1.1.1. Ограничения

Система разрабатывается для работы в европейской части РФ: Центральном, Северо-западном (кроме Калининградской области), Приволжском и Южным федеральных округах, определяемой сетевой и транспортной доступностью.

Не рассматриваются проблемы взаимодействия с зарубежными пользователями и продажа/покупка в валюте, отличной от российского рубля.

Не рассматриваются проблемы и риски, связанные с финансовой (налоговой) отчётностью владельца системы, и соответствующий функционал.

1.1.2. Условия

Для разработки и внедрения системы необходимы:

- Наличие договора(ов) с агрегатором(ами) платежей и средств использования системы электронных платежей этого агрегатора (API).
- Наличие договора(ов) с логистическими компаниями по доставке товаров от продавцов к покупателям с возможностью автоматического отслеживания прохождения доставки заказа с использованием средств, предоставляемых логистическими компаниями (API). При этом предполагается указание сервиса при отправке продавцом товара покупателю в пункте приёма товара логистической компанией.
- Регистрация у провайдеров систем двухфакторной аутентификации.
- Регистрация компании-оператора системы в качестве оператора персональных данных.

Глава 2. Требования

2.1. Функциональные требования

1. Возможность для продавца создавать, редактировать, публиковать, архивировать (удалять из видимости), восстанавливать и удалять карточки товаров/услуг.
2. Возможность для покупателя просматривать объявления, осуществлять по ним классификационный, атрибутивный и полнотекстовой поиск.
3. Поддержка медиаконтента (изображений, видео), описаний, атрибутов, тегирования товаров и услуг.
4. Наличие рекомендательной системы, учитывающей историю покупателя и подбирающей сходные объявленя; также готовящей начальный список рекомендаций при регистрации покупателя в системе.
5. Возможность для покупателя оформлять заказы средствами сервиса с фиксацией условий сделки и резервированием заказанного товара.
6. Информирование продавца о новом заказе.
7. Возможность для покупателя вести диалог с продавцом по конкретным объявлениям и с сервисом по условиям работы сервиса.
8. Средства регистрации продавцов и покупателей и их контактов на площадке сервиса, с возможностью идентификации продавцов.
9. Модерация объявлений перед публикацией, в том числе, автоматическая.
10. Возможность для зарегистрированного покупателя вести самостоятельную одноуровневую классификацию товаров (целевые списки).
11. Создание механизмов оплаты заказов покупателями средствами сервиса с использованием систем электронных платежей внешнего агрегатора, с учётом удержания комиссии в пользу сервиса и оформлением платежей в соответствии с законодательством.
12. Публикация средствами сервиса единой публичной оферты (условий обслуживания) для продавцов и покупателей.
13. Возможность категоризации продавцов и покупателей.
14. Возможность создания специальных условий работы с сервисом (размер комиссии, сроки её поступления) для указанных категорий продавцов или отдельных продавцов.
15. Возможность отслеживания доставки товаров покупателям средствами сервиса с использованием средств, предоставляемых для этого логистическими компаниями, ответственными за доставку.
16. Формирование отчётности для продавцов и администрации сервиса о сделках, начислениях и выплатах, с возможностью фильрации по категориям товаров и продавцов, периодам, статусам.

2.2. Нефункциональные требования

2.2.1. Производительность

1. Время отклика интерфейса сервиса на действия пользователя не более 2-х секунд.
2. Оформление заказа после выбора не более, чем за 4 шага.
3. Доступность ключевых функций не более, чем за 3 клика от главной страницы.
4. Время загрузки страниц каталога не более 3-х секунд при 90 процентили.
5. Загрузка карточки товара: не более 1,5 секунд
6. Обработка поискового запроса: не более 5 секунд
7. Проведение платежной транзакции (до выхода на интерфейс работы с платежной системой): не более 5 секунд
8. Поддержка не менее 10 000 одновременных пользователей
9. Обработка не менее 1000 транзакций в минуту
10. Размещение не менее 500 объявлений в час
11. Интерфейс на русском языке.
12. При указаниях времени поддержка часовых поясов регионов присутствия с автоматическим определением местоположения пользователя и возможностью его ручного изменения пользователем.

2.2.2. Надежность и доступность

1. Целевой уровень доступности системы: 99,5% (uptime)
2. Плановые технические работы: не более 8 часов в месяц в ночное время
3. Резервирование критичных компонентов (база данных, платёжный шлюз)
4. Автоматическое восстановление после сбоя в течение 15 минут
5. Отсутствие потери данных о транзакциях и объявлениях
6. Время восстановления после критического сбоя (RTO): не более 4 часов
7. Точка восстановления данных (RPO): не более 1 часа

2.2.3. Масштабирование и развитие

1. Возможность добавления серверов приложений без простоя
2. Поддержка физически распределённой архитектуры
3. Поддержка географически распределённых дата-центров
4. Поддержка роста базы объявлений до 100 млн записей
5. Поддержка базы пользователей до 50 млн учётных записей

2.2.4. Безопасность

1. Двухфакторная аутентификация для финансовых операций
2. Разграничение прав доступа по ролям (покупатель, продавец, администратор, модератор)
3. Сессии пользователей с автоматическим завершением после 30 минут неактивности
4. Шифрование персональных данных в базе данных (AES-256)
5. Шифрование передачи данных (TLS 1.2 и выше)
6. Защита от DDoS-атак
7. Защита от SQL-инъекций и XSS-атак
8. Ограничение частоты запросов (rate limiting)
9. Логирование всех операций с финансовыми данными
10. Соответствие требованиям 152-ФЗ о персональных данных
11. Соответствие регуляторным требованиям при осуществлении финансовых операций и учёте их в системе.

2.2.5. Эргономика и удобство использования

1. Адаптивный дизайн для desktop, планшетов и мобильных устройств
2. Поддержка браузеров: Chrome, Firefox, Safari, Edge (последние 2 версии)
3. Поддержка мобильных платформ: iOS 14+, Android 10+
4. Размещение объявления не более чем в 5 шагов
5. Оформление заказа не более чем в 4 шага
6. Доступность ключевых функций не более чем за 3 клика от главной страницы
7. Интерфейс на русском языке
8. Поддержка часовых поясов регионов присутствия
9. Форматы даты, времени и валюты в соответствии с российскими стандартами

2.2.6. Сопровождаемость

1. Централизованное логирование всех компонентов системы
2. Мониторинг ключевых метрик производительности в реальном времени
3. Автоматические уведомления администраторов о критических событиях
4. Техническая документация на архитектуру и API
5. Руководства пользователя и администратора
6. Актуализация документации при каждом обновлении
7. Возможность обновления компонентов без полной остановки системы
8. Откат к предыдущей версии в случае проблем при обновлении

2.2.7. Интеграции

1. Совместимость с платёжными системами (СБП, банковские карты)
2. Совместимость с сервисами доставки
3. REST API для интеграции со сторонними системами

2.2.8. Документирование

1. Наличие документации для покупателей в форме многостраничного структурированного веб-документа, доступного для скачивания в pdf-файл.
2. Наличие документации для продавцов в форме многостраничного структурированного веб-документа, доступного для скачивания в pdf-файл.
3. Наличие документации администратора, включающей вопросы конфигурирования, настройки, взаимодействия с внешними системами.
4. Наличие в документации разделов FAQ (friquently asked questions, часто задаваемые вопросы). В рамках сопровождения уточнение и дополнение этих разделов по результатам реальзных потребностей пользователей.

2.3. Риски

2.3.1. Инфраструктурные риски

ID	Риск	Влияние	Описание
RSK01_01	Отказ API Gateway	Высокое	Полная недоступность системы для пользователей при единой точке входа. Приводит к остановке всех бизнес-процессов.
RSK01_02	Потеря данных заказов	Критичное	Необратимая потеря информации о сделках, невозможность выполнения обязательств, финансовые потери.
RSK01_03	Недоступность платежных провайдеров	Высокое	Блокировка процесса оформления заказов, потеря продаж, недовольство пользователей. При пиковых нагрузках.
RSK01_04	Недоступность транспортных компаний	Среднее	Ошибки и задержки информирования пользователей, репутационные потери.
RSK01_05	Несогласованность данных о наличии товаров	Высокое	Продажа отсутствующих товаров, отмены заказов, негативный пользовательский опыт, репутационные потери.
RSK01_06	Перегрузка системы в пиковые периоды	Высокое	Падение производительности или полный отказ системы во время распродаж, акций, праздничных периодов.

RSK01_07	Потеря сообщений в шине событий	Среднее	Нарушение бизнес-процессов, требующих асинхронной обработки (например, обновление инвентаря после покупки).
RSK01_08	Недоступность базы данных	Критичное	Остановка всех операций системы, так как большинство сервисов зависят от доступа к данным.

2.3.2. Бизнес-риски

ID	Риск	Влияние	Описание
RSK02_01	Двойное списание средств	Критичное	Разнообразные фатальные последствия. Возникает при нарушении идемпотентности операций.
RSK02_02	Ошибки в расчете комиссий	Критичное	Финансовые потери платформы (недополучение комиссии) или продавцов (завышенные списания), конфликты с контрагентами.
RSK02_03	Некорректные выплаты продавцам	Критичное	Финансовые потери, судебные разбирательства, репутационный ущерб, отток продавцов с платформы.
RSK02_04	Нарушение финансовой отчетности	Высокое	Невозможность корректного учета, налоговые проблемы, санкции со стороны контролирующих органов.
RSK02_05	Потеря трек-номеров доставки	Среднее	Невозможность отслеживания заказов, увеличение нагрузки на поддержку, недовольство покупателей.
RSK02_06	Задержки синхронизации с внешними системами	Среднее	Некорректное отображение статусов доставки, остатков товаров у продавцов, курсов валют для оплаты.
RSK02_07	Недоставка критичных уведомлений	Среднее	Пользователи не получают информацию о статусе заказов, подтверждении оплаты, что ведет к росту обращений в поддержку.
RSK02_08	Нарушение ФЗ-152 "О персональных данных"	Критичное	Административная и уголовная ответственность, крупные штрафы, приостановка деятельности.

2.3.3. Риски интеграций

ID	Риск	Влияние	Описание
RSK03_01	Изменение API внешних сервисов без обратной совместимости	Высокое	Внезапный сбой интеграций с платежными системами, службами доставки, требующий срочного исправления кода.

ID	Риск	Влияние	Описание
RSK03_02	Превышение лимитов запросов к внешним API	Среднее	Блокировка доступа к критичным сервисам (платежи, доставка) до начала следующего расчетного периода.
RSK03_03	Утрата API-ключей	Высокое	Компрометация интеграций, возможные финансовые потери при утечке платежных ключей.

2.3.4. Операционные риски

ID	Риск	Влияние	Описание
RSK04_01	Недостаточный мониторинг	Высокое	Позднее обнаружение инцидентов, увеличение времени простоя, невозможность предотвращения проблем.
RSK04_02	Отсутствие плана аварийного восстановления (DRP)	Критичное	Длительное время восстановления после серьезных сбоев, несистематические действия при инцидентах.
RSK04_03	Недостаточная емкость хранилищ	Среднее	Прекращение работы системы при исчерпании дискового пространства (особенно для логов и медиафайлов).

2.3.5. Риски безопасности

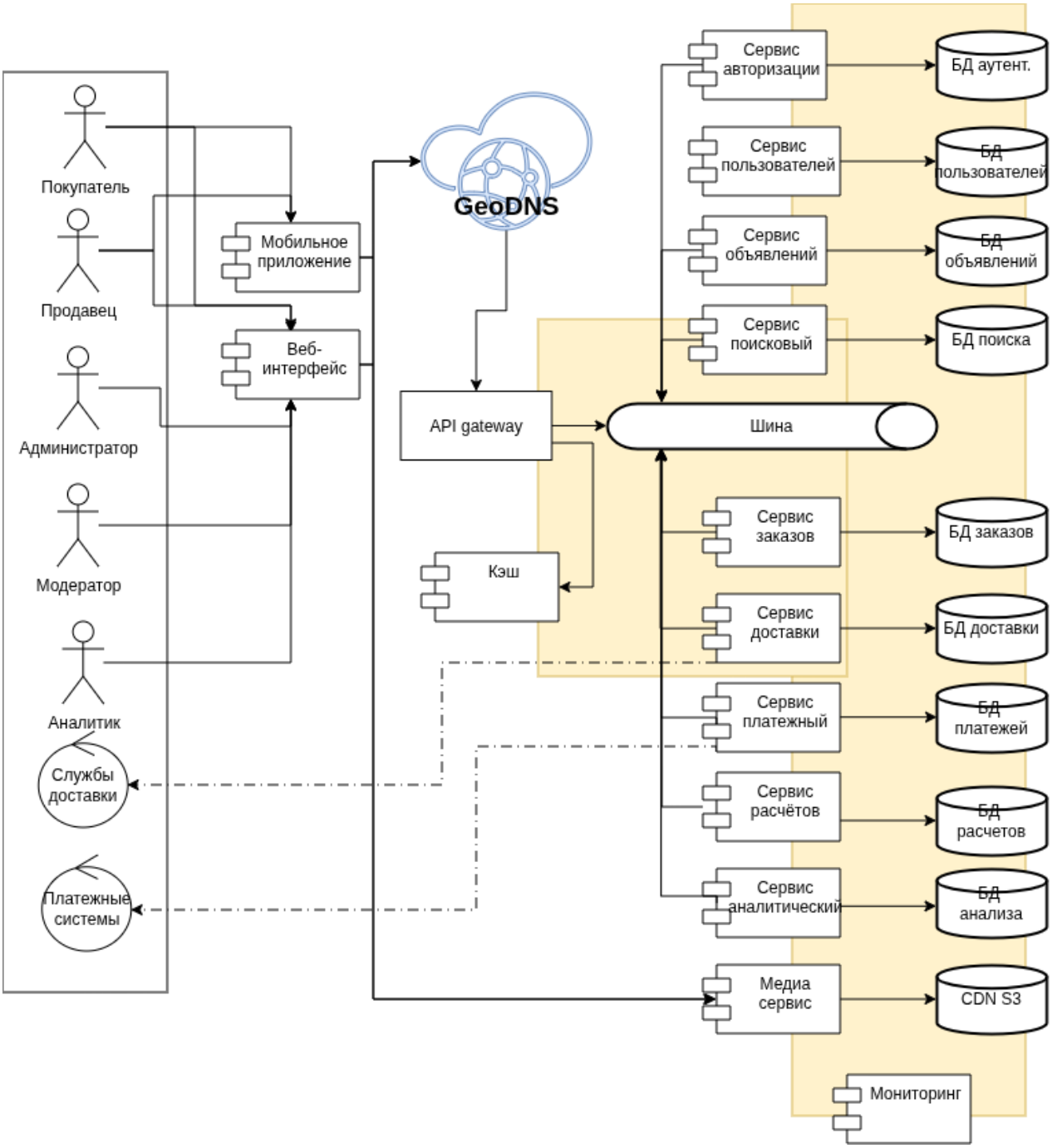
ID	Риск	Влияние	Описание
RSK05_01	SQL-инъекции и утечки данных	Критичное	Компрометация персональных и платежных данных пользователей, юридическая ответственность, репутационные потери.
RSK05_02	DDoS-атаки на публичные точки доступа	Высокое	Недоступность сервиса для легитимных пользователей, финансовые потери от простоя, репутационные потери.
RSK05_03	Межсайтовый скриптинг (XSS)	Высокое	Кража сессий пользователей, модификация контента на стороне клиента, атаки на других пользователей.
RSK05_04	Подделка межсайтовых запросов (CSRF)	Среднее	Неавторизованное выполнение действий от имени аутентифицированного пользователя.
RSK05_05	Инсайдерские угрозы	Критичное	Умышленное или неумышленное причинение вреда системе сотрудниками с доступом к production-окружению.

2.3.6. Риски непрерывности

ID	Риск	Влияние	Описание
RSK06_01	Несоответствие резервных копий	Критичное	Невозможность восстановления данных после сбоя (RSK01_02, RSK01_08) из-за ошибок в процессе бэкапирования.
RSK06_02	Истечение сроков действия SSL-сертификатов	Высокое	Прекращение доступа к HTTPS-эндпоинтам (API Gateway, сервисы), блокировка работы клиентов.
RSK06_03	Утечка учетных данных персонала	Критичное	Компрометация производственного окружения через учетные записи сотрудников (усугубляет RSK05_05).

Глава 3. Концептуальная архитектура

3.1. Схема концептуальной архитектуры



3.2. Описание концептуальной архитектуры

Система имеет сервисную архитектуру и реализуется в виде укрупнённых сервисов с использованием:

- синхронных взаимодействий (REST/gRPC)
- асинхронных взаимодействий (Kafka)
- polling для пользовательских сообщений (без realtime/WebSocket)

- для поддержки географического охвата георасределения через GeoDNS (Москва / Краснодар), а также для доставки медиа-контента поддерживает CDN с узлами в Санкт-Петербурге и Казани.

Система состоит из следующих сервисов:

1. Сервис авторизации, опирающийся на Базу Данных Авторизации и поддерживающий аутентификацию, авторизацию и поддержание сессий работы пользователей.
2. Сервис профилей пользователей, опирающийся на Базу Данных Пользователей и поддерживающий информацию о них, целевые списки (wish-листы) пользователей, получаемую информацию о поведении.
3. Сервис объявлений, включающий Базу Данных Объявлений, содержащую основные данные объявлений. Поддерживает актуальное состояние и резервирование товаров по объявлениям. Через него осуществляется обмен сообщениями между покупателями и продавцами.
4. Сервис поиска, связанный с сервисом объявлений, и использующий отдельную или совмещённую с сервисом объявлений Базу Данных, настроенную для параметрического и текстового поиска. Также реализует функционал рекомендаций покупателям.
5. Сервис заказов, включающий Базу Данных Заказов, содержащую информацию о товарном, логистическом и финансовом состоянии заказов
6. Сервис доставки отвечает за связь с API логистических компаний и формирование (при необходимости - моделирование) данных о транспортировке заказов.
7. Сервис платежей отвечает за обработку платежей через связь с API платежных сервисов.
8. Сервис расчётов, учитывающий финансовую часть сделок и комиссию торговой площадки.
9. Аналитический сервис предоставляет отчётную информацию о потоке заказов и финансовых показателях системы, рассылает уведомления пользователям.
10. Медиа-сервис поддержки CDN.

Сервисы взаимодействуют между собой через общий брокер сообщений ("шину данных"). С внешними поставщиками данных (платёжные и логистические сервисы) соответствующие сервисы системы взаимодействуют напрямую. Отдельные сервисы также могут взаимодействовать синхронно напрямую, если этого требует логика бизнес-процесса. Такие взаимодействия будут специально описаны в соответствующих частях.

Пользователи взаимодействуют с системой через веб-интерфейс или мобильное приложение (фронтенд). Фронтенд и, при необходимости, другие внешние информационные системы обмениваются с системой данными посредством API с единой точкой входа.

Информирование пользователей осуществляется через публикацию сообщений в личных кабинетах. Диалоги между покупателями и продавцами также привязываются к сервису объявлений и ведутся в асинхронном режиме.

3.3. Пользователи решения

Для работы с системой устанавливаются следующие роли пользователей:

3.3.1. Внешние пользователи

1. Покупатель - пользователь, которому доступен поиск товаров по объявлениям, просмотр объявлений, формирование и оплата заказов, получение отчётов по своим покупкам и оплатам. Для осуществления взаимодействия с ним идентифицируется в системе, по крайней мере, адресом электронной почты или номером телефона.
2. Продавец - пользователь, которому доступно размещение, редактирование и архивирование (логичкское удаление) объявлений (в части редактирования и архивирования - только для своих объявлений), отчёты по своим объявлениям и их оплате. Идентифицируется адресом(ами) электронной почты, номером(ами) телефона и реквизитами для осуществления платежей.
    - Гость — незарегистрированнй (неавторизованный) пользователь, просматривает каталог без оформления заказов

3.3.2. Внутренние пользователи

1. Модератор - пользователь, контролирующий контент и взаимодействующий с внешними пользователями
2. Администратор - пользователь, осуществляющий технические настройки и поддержание работоспособности системы.
3. Аналитик - пользователь, анализирующий данные и формирующий отчеты

3.3.3. Внешние системы

- Платежные системы — обрабатывают платежи
- Службы доставки — обрабатывают доставку заказов

3.4. Внешние интеграции

3.4.1. Интеграция с платёжными агрегаторами

- Взаимодействие с платёжными агрегаторами поддерживается через предоставляемые ими API с использованием протокола HTTPS.
- Должны поддерживаться следующие способы оплаты: банковские карты (Visa, Mastercard, МИР), электронные кошельки, СБП
- Система должна получать уведомления о статусе платежа (успешный, отклоненный, ожидает подтверждения) в режиме реального времени через webhook.
- Должна быть реализована возможность возврата средств покупателю через API агрегатора.
- Система должна обеспечивать выплаты продавцам с учетом комиссии маркетплейса.

- Должно обеспечиваться логирование всех транзакций с сохранением идентификаторов платежей в обеих системах.
- При недоступности платежного агрегатора система должна информировать пользователя о временной невозможности совершить платеж.
- Система должна поддерживать механизм повторных попыток при сбоях (retry mechanism) с экспоненциальной задержкой.
- Взаимодействие платежного сервиса с API агрегатора осуществляется в синхронном режиме.

Передаваемые данные:

Исходящие: сумма платежа, идентификатор заказа, данные покупателя (email, телефон), URL для возврата после оплаты

Входящие: статус платежа, идентификатор транзакции, timestamp операции, детали ошибки (при наличии)

3.4.2. Интеграция с транспортной компанией

Назначение: оформление доставки заказов, отслеживание статуса доставки.

- Система должна автоматически создавать заявку на доставку через API транспортной компании при подтверждении заказа
- Должен рассчитываться предварительный срок и стоимость доставки на основе данных о весе, габаритах товара и адресе получателя
- Система должна получать актуальные данные о тарифах доставки для отображения покупателю при оформлении заказа
- Должна обеспечиваться генерация транспортных накладных и печатных форм для маркировки отправлений
- Система должна получать обновления статуса доставки (принято на склад, в пути, доставлено, возврат) через webhook или polling с интервалом не более 30 минут
- Должна быть реализована возможность вызова курьера для забора товара от продавца
- Система должна поддерживать отмену доставки и оформление возврата через API транспортной компании
- При недоступности API система должна использовать последние загруженные тарифы и предупреждать администратора о необходимости ручной обработки

Передаваемые данные:

Исходящие: параметры отправления (вес, габариты, объявленная стоимость), адреса отправителя и получателя, контактные данные, идентификатор заказа в системе маркетплейса

Входящие: трек-номер отправления, статус доставки, история перемещения, стоимость доставки, дата доставки (фактическая/планируемая)

3.5. Потоки данных и высокоуровневое взаимодействие

3.5.1. Информационные потоки

3.6. Потоки данных и высокоуровневое взаимодействие

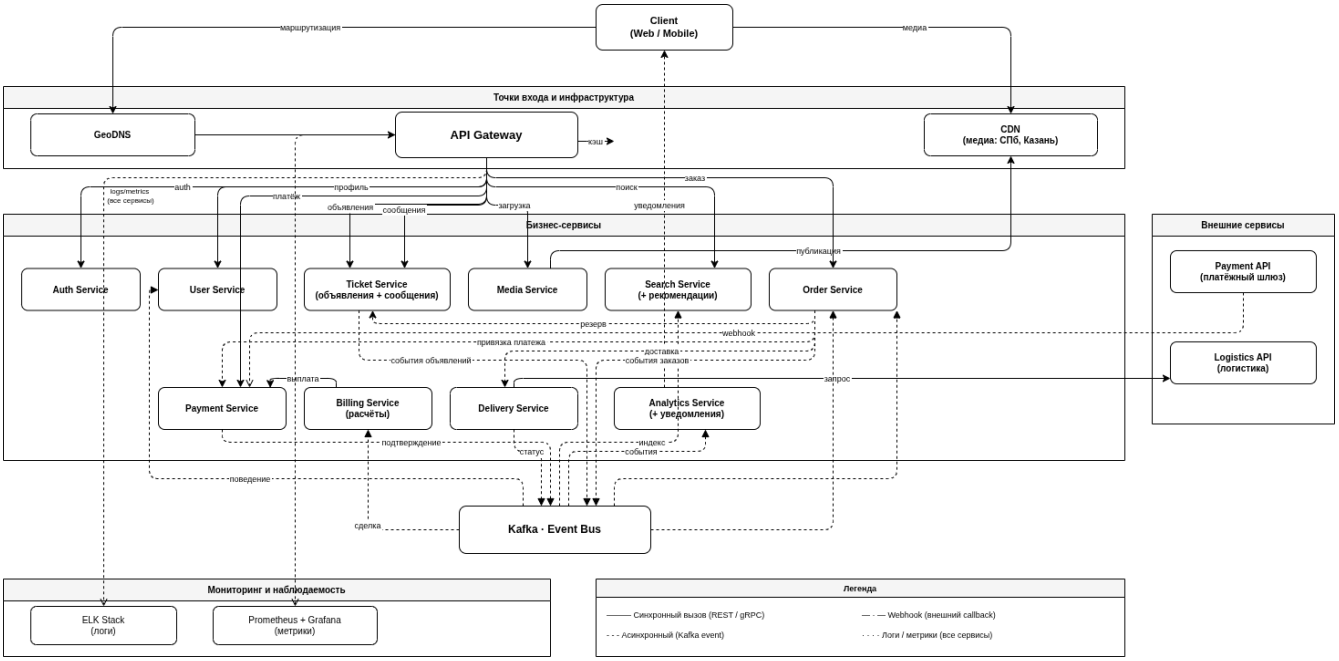
Раздел описывает движение данных между компонентами системы. Указаны источник, приёмник, тип данных и способ передачи.

3.6.1. Потоки данных

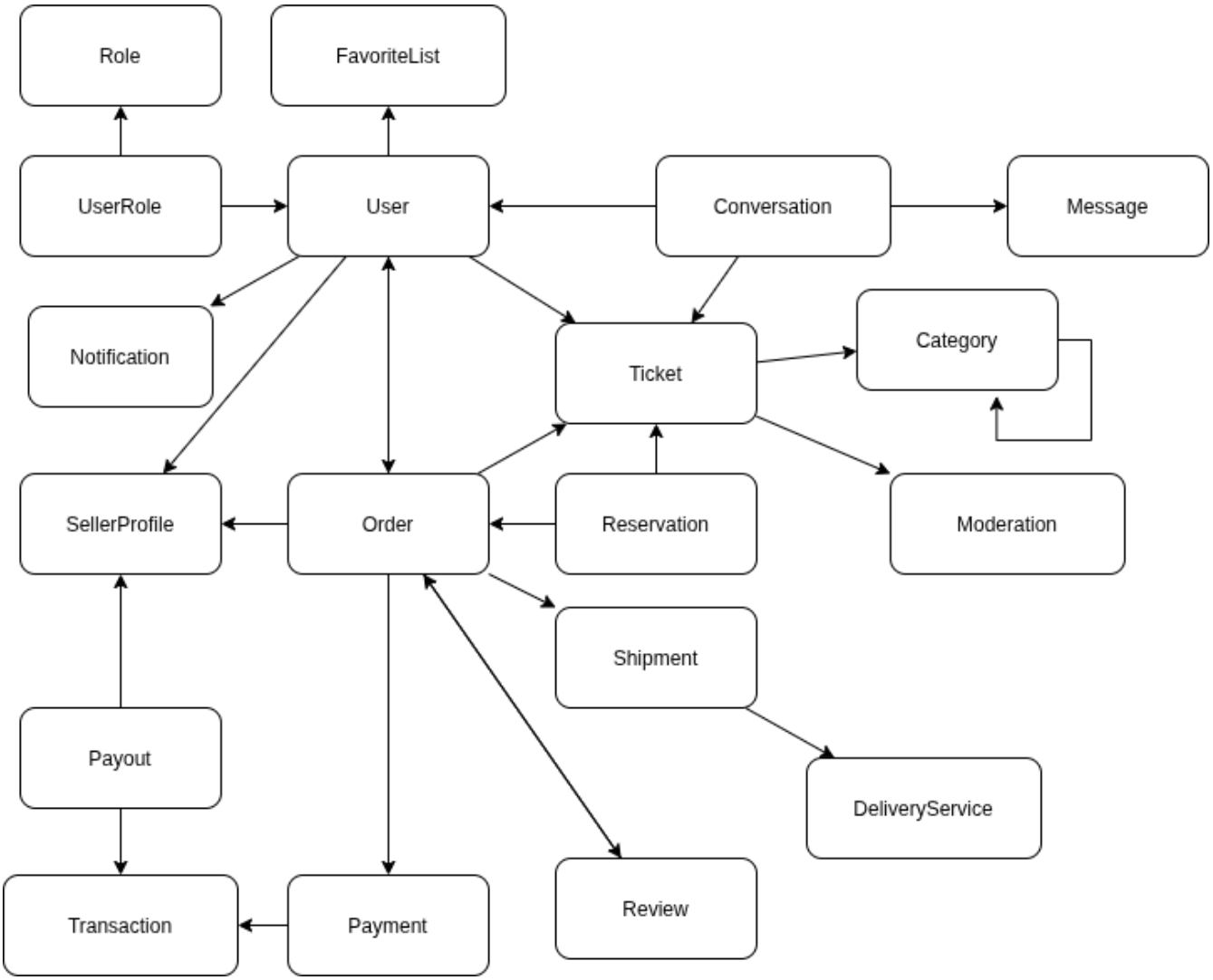
ID	Источник → Приёмник	Тип данных	Способ
IF-01	Client → GeoDNS → API Gateway	HTTP request	Sync
IF-02	Client → CDN	Media	Sync
IF-03	API Gateway → Auth Service	JWT token, session_id	Sync
IF-04	Auth Service → PostgreSQL	Credentials (login, password hash)	Sync
IF-05	Auth Service → Redis	Session, token	Sync
IF-06	API Gateway → Redis	Session lookup	Sync
IF-07	API Gateway → User Service	Profile data	Sync
IF-08	User Service → PostgreSQL	User data	Sync
IF-09	API Gateway → Ticket Service	Ticket data	Sync
IF-10	Ticket Service → PostgreSQL	Ticket data	Sync
IF-11	Ticket Service → S3	Media binary	Sync
IF-12	S3 → CDN	Media object	Async
IF-13	Ticket Service → Kafka	ticket.events	Async
IF-14	Kafka → Search Service	ticket.events	Async
IF-15	Search Service → ElasticSearch	Index data	Sync
IF-16	API Gateway → Search Service	Search query	Sync
IF-17	API Gateway → Order Service	Order data	Sync
IF-18	Order Service → PostgreSQL	Order data	Sync
IF-19	Order Service → Ticket Service	Reserve request	Sync

ID	Источник → Приёмник	Тип данных	Способ
IF-20	Order Service → Payment Service	Payment request	Sync
IF-21	Order Service → Delivery Service	Delivery request	Sync
IF-22	Payment Service → PostgreSQL	Payment data	Sync
IF-23	Payment Service → External Payment API	Payment request	Sync
IF-24	External Payment API → Payment Service	Payment status webhook	Async
IF-25	Payment Service → Kafka	payment.events	Async
IF-26	Kafka → Order Service	payment.events	Async
IF-27	Delivery Service → PostgreSQL	Delivery data	Sync
IF-28	Delivery Service → External Logistics API	Shipment request	Sync
IF-29	External Logistics API → Delivery Service	Delivery status webhook	Async
IF-30	Delivery Service → Kafka	delivery.events	Async
IF-31	Kafka → Order Service	delivery.events	Async
IF-32	Order Service → Kafka	order.events	Async
IF-33	Kafka → Billing Service	order.events	Async
IF-34	Billing Service → PostgreSQL	Billing data	Sync
IF-35	Billing Service → Payment Service	Payout request	Sync
IF-36	Ticket Service → Kafka	message.events	Async
IF-37	Kafka → Analytics Service (ClickHouse)	ticket.events, order.events, payment.events, delivery.events, message.events	Async

3.6.2. Схема взаимодействия



3.7. Модель данных



User — пользователь системы (покупатель или сотрудник платформы).

Role — роль пользователя (покупатель, продавец, администратор, модератор, аналитик).

UserRole — связка пользователя и его ролей.

SellerProfile — профиль продавца с дополнительными данными (рейтинг, тип, реквизиты).

Ticket — объявление о продаже товара или услуги.

Moderation — статус и результат модерации объявления.

Conversation — диалог между пользователями (набор сообщений), по объявлению.

Message — отдельное сообщение в диалоге.

Order — заказ, связывает покупателя, продавца и объявление.

Payment — операция оплаты заказа покупателем.

Payout — операция выплаты средств продавцу.

Transaction — финансовая проводка (движение денег внутри системы).

Shipment — доставка заказа с трекингом.

DeliveryService — служба доставки (внешний провайдер).

Category — категория (рубрика) объявлений.

FavoriteList — список избранных объявлений пользователя.

Notification — уведомление пользователю о событиях системы.

Review — отзыв пользователя о другом пользователе или сделке.

3.8. Расчеты показателей решения

3.8.1. Исходные данные

Население европейской части РФ оценивается в ~100 млн человек.

Источник: https://ru.wikipedia.org/wiki/Европейская_часть_России

Доля интернет-пользователей: ~90%

Источник: https://sky.pro/wiki/analytics/skolko-lyudej-v-rossii-polzuyutsya-internetom-statistika-i-fakty/

Потенциальная аудитория: 100 млн × 80% = 90 млн пользователей

Оптимистическая оценка доли пользователей системы: 5%

MAU: 90 млн × 5% = 4,5 млн пользователей

DAU/MAU: 20% https://cleverdata.ru/blog/statii/dau-wau-mau-metriki?utm_source=chatgpt.com

DAU: 4,5 млн × 20% = 1 млн. пользователей

3.8.2. Распределение пользователей

- значительная часть пользователей сосредоточена в Москве и Санкт-Петербурге
- центральные регионы и Поволжье дают сопоставимую, но более распределённую нагрузку
- южные и северные регионы менее плотные, но дают заметную долю пользователей

3.8.3. Пользовательская активность

По открытым исследованиям:

- около 50% пользователей заходят на e-commerce ресурсы ежедневно
- среднее время использования составляет около 25 минут в день
- пользователи совершают несколько сессий и просмотров за одно посещение

Исходя из этого, принято:

- 2–4 сессии в день
- 5–10 действий на сессию

Итого — порядка 10–20 действий на пользователя в сутки (принято 15).

Источник: https://adindex.ru/news/researches/2024/11/13/327315.phtml https://mpgo.ru/news/novosti-industrii/data_insight_obem_e_com_rynka_v_rossii_vyros_na_39/

Принятые значения:

- 15 действий на пользователя в сутки
- 5 запросов на действие

Итого: 75 запросов на пользователя в сутки

3.8.4. Суточная нагрузка

Общий объём запросов: 1 000 000 × 75 = 75 млн запросов/сутки

Средняя нагрузка: 75 млн / 86400 ≈ 868 req/c ~ 900 req/c

3.8.5. Пиковая и всплесковая нагрузка

Коэффициент пиковости: 5

Пиковая нагрузка: 900 × 5 ≈ 4340 req/c ~ 4500 req/c

Коэффициент всплеска (акции, сезонность): 2

Всплесковая нагрузка: 4500 × 2 ≈ 9000 req/sec

3.8.6. Публикация объявлений

Исходя из данных по Авито (2,6%), принимается консервативная оценка доли публикующих пользователей:

1% DAU

Новые объявления: 1 000 000 × 1% = 10 000 объявлений/сутки

Срок жизни объявления принимается: 60 дней

Активные объявления: 10 000 × 60 = 600 000

Архив (1 год): 10 000 × 365 ~ 3,7 млн

3.8.7. Медиа-данные

Параметры медиа приняты, оценочно:

- 5 изображений на объявление
- средний размер изображения — 500 KB
- 20% объявлений содержат видео
- средний размер видео — 10 MB

Активные данные:

Изображения: 600 000 × 5 × 0.5 MB = 1.5 TB

Видео: 600 000 × 20% × 10 MB = 1,2 TB

Итого: = 2.7 TB

Архив: ≈ 12 TB

3.8.8. Структурированные данные

Размер записи объявления: 3 KB

Активные объявления: 600 000 × 3 KB ≈ 1.75 GB

Архив: 3,7 млн × 3 KB ≈ 11,1 GB

Пользователи: 4,5 млн × 2 KB ≈ 9 GB

Суммарно: ≈ 21 GB

С учётом индексов и служебных данных: ≈ до 100 GB

3.8.9. Сетевой трафик

Средний размер ответа: 200 KB

Суточный трафик: 75 млн × 200 KB ≈ 15 TB/сутки

Средний поток: ≈ 175 MB/sec

Пиковый поток: ≈ 875 MB/sec

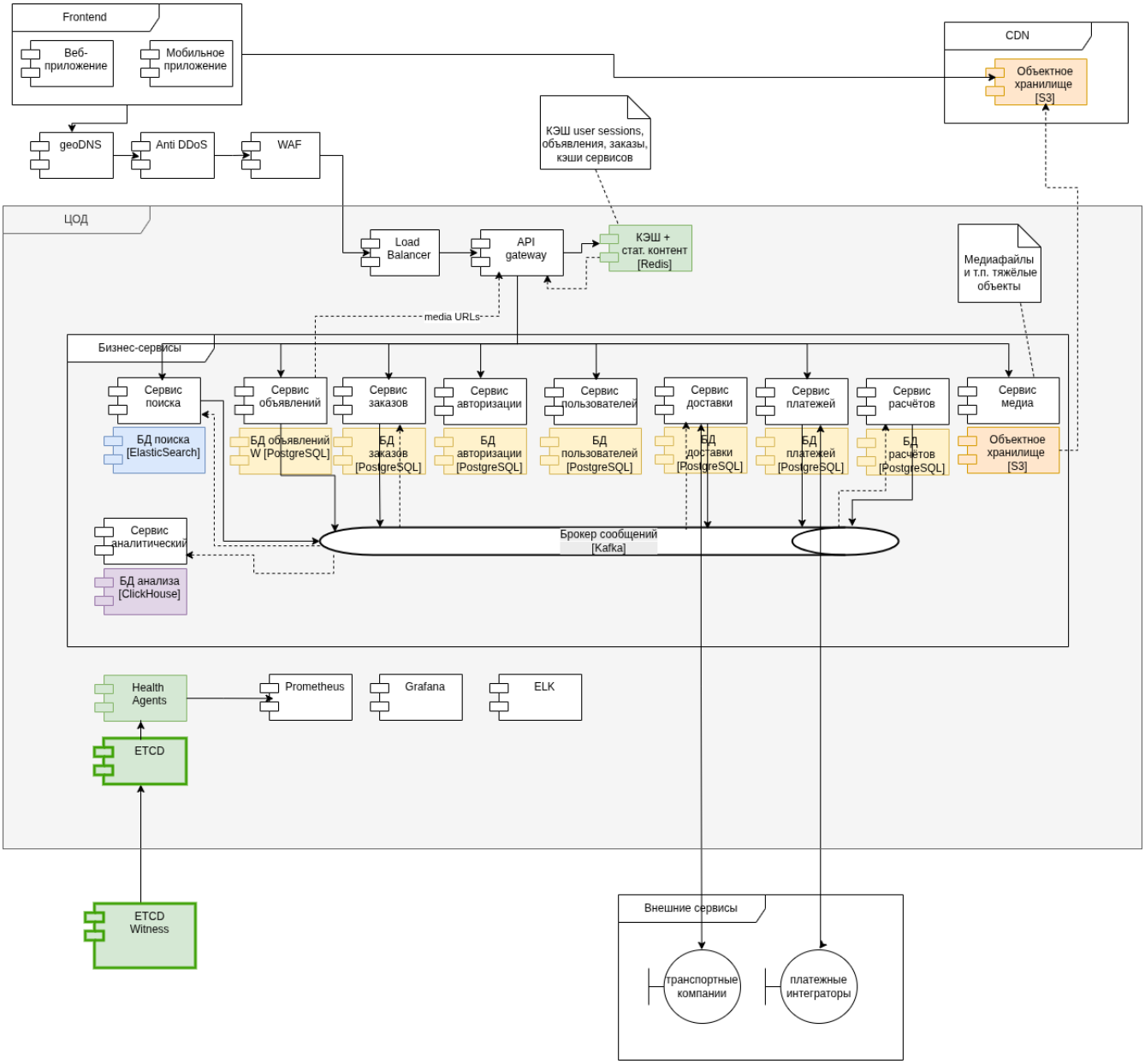
3.8.10. Вывод

Ожидаемые параметры нагрузки системы:

- DAU: ~1 млн пользователей
- MAU: ~5 млн пользователей
- Средняя нагрузка: ~900 req/sec
- Пиковая нагрузка: ~4500 req/sec
- Всплесковая нагрузка: до ~9000 req/sec
- Активные объявления: ~600 тыс
- Объём медиа: ~2.7 TB (активные)
- Сетевой трафик: ~15 TB/сутки

Нагрузка распределяется неравномерно, с концентрацией до 40% в регионах с высокой плотностью населения.

3.9. Схема архитектуры решения



3.10. Observability

3.10.1. Показатели

Бизнес-процессы

- Количество активных пользователей, сессий
- Создание/просмотры объявлений
- Создание/оплата/доставка заказов
- Успешность платежей, средний чек
- Скорость работы поиска, релевантность результатов

Техническое состояние

- Доступность всех сервисов (API Gateway, авторизация, поиск, заказы, платежи)

- Время ответа каждого сервиса
- Загрузка процессора, памяти, дисков на всех серверах
- Состояние баз данных (PostgreSQL, Elasticsearch, Redis)
- Очереди сообщений в Kafka (нет накопления)
- Работа внешних API (платежные системы, службы доставки)

3.10.2. Средства

Дашборды (Grafana)

- **"Общее состояние"**: Сводка по всем критичным сервисам, общее количество заказов, ошибок.
- **"Финансы"**: Статусы платежей, успешные/неуспешные транзакции, среднее время проведения.
- **"Поиск и каталог"**: Время ответа поиска, количество запросов, загрузка Elasticsearch.
- **"Инфраструктура"**: Загрузка серверов, состояние баз данных, использование кэша.

Автоматическое оповещение

- Когда любой сервис недоступен более 1 минуты
- При росте количества ошибок платежей выше 5%
- Если время ответа поиска превышает 2 секунды
- При заполнении диска на сервере БД более 85%
- При сбоях в работе брокера сообщений или кэша
- Все алерты поступают в единый Telegram-чат с приоритетами (критический/предупреждение)

3.10.3. Инструменты диагностики

- **Логи (агенты + ELK)**: Логи из всех сервисов хранятся централизованно.
- **Трассировка (Jaeger)**: Путь выполнения запросов.
- **Инфраструктурные метрики (Prometheus)**: Детальные показатели по каждому серверу и базе данных в реальном времени.

3.11. Реагирование на инциденты

3.11.1. Роли и ответственность

Роль	Ответственность	Контакт
Инженер сопровождения	Координация действий, коммуникация со стейкхолдерами, фиксация таймлайна.	Дежурный архитектор / Владелец продукта

Роль	Ответственность	Контакт
Владелец продукта	Диагностика, принятие технических решений, руководство инженерами.	Дежурный разработчик
Специалист по безопасности	Анализ угроз безопасности, изоляция compromised узлов, работа с 152-ФЗ.	CISO
Пресс-секретарь	Внешние коммуникации (пользователи, партнеры), подготовка уведомлений.	PR-менеджер

3.11.2. Мониторинг и обнаружение

Система мониторинга должна покрывать следующие метрики для своевременного обнаружения рисков:

- **Инфраструктура (RSK01_01, RSK01_06, RSK01_08):**
  - Доступность API Gateway (соотношение кодов откликов HTTP 200/5xx).
  - Задержка ответов сервисов сервисов (p95, p99).
  - Утилизация CPU/RAM/Disk (>85%).
- **Бизнес-процессы (RSK02_01, RSK02_02, RSK02_03):**
  - Количество успешных/неуспешных платежей в минуту (аномалии >20%).
  - Расхождение сумм транзакций и заказов (периодическая автоматическая сверка).
  - Количество отмен заказов по причине «нет товара» (RSK01_05).
- **Безопасность (RSK05_01, RSK05_02, RSK06_02):**
  - Количество неудачных попыток авторизации.
  - Трафик на входе (DDoS-детекция).
  - Срок действия SSL-сертификатов (алерт за 14 и 3 дня).
  - Доступ к чувствительным данным (логи аудита).

3.11.3. Классификация инцидентов

Уровень	Критерии	Время реакции	Время восстановления
ILVL-1 (Критический)	Полная недоступность системы, потеря данных, утечка ПДн (RSK01_01, RSK01_02, RSK02_08).	15 минут	Непрерывно до устранения
ILVL-2 (Высокий)	Деградация функционала (оплата, доставка), ошибки >10% запросов (RSK01_03, RSK01_05).	30 минут	4 часа

Уровень	Критерии	Время реакции	Время восстановления
ILVL-3 (Средний)	Частичные ошибки, проблемы с медиа/поиском, задержки уведомлений (RSK01_04, RSK02_07).	2 часа	24 часа

3.11.4. Сценарии действий (Runbooks)

Отказ API Gateway / DDoS-атака (RSK01_01, RSK05_02)

1. **Обнаружение:** Алерт по росту 5xx ошибок или аномальному трафику.
2. **Действия:**
   a. Активация ограничений «Под нагрузкой» на firewall/API Gateway.
   b. Масштабирование входных нод (auto-scaling).
   c. При необходимости — включение статической страницы «Технические работы».
3. **Восстановление:** Анализ логов атаки, обновление правил firewall.

Сбой платежного шлюза (RSK01_03, RSK02_01)

1. **Обнаружение:** Падение конверсии оплаты, алерт от платежного провайдера.
2. **Действия:**
   a. Переключение трафика на резервный платежный провайдер (если доступно).
   b. Временная приостановка приема новых платежей для избежания двойных списаний (RSK02_01).
   c. Ручная обработка зависших транзакций.
3. **Восстановление:** Сверка реестров с провайдером, возобновление работы.

Потеря данных / Отказ БД (RSK01_02, RSK01_08, RSK06_01)

1. **Обнаружение:** Ошибки подключения к БД, алерт целостности реплик.
2. **Действия:**
   a. Переключение на горячую реплику (Failover).
   b. При полной потере — восстановление из последней валидной резервной копии (проверка RSK06_01).
   c. Блокировка записи данных до восстановления целостности.
3. **Восстановление:** Проверка консистентности заказов и балансов пользователей.

Утечка персональных данных (RSK02_08, RSK05_01)

1. **Обнаружение:** Аномальный доступ к БД, сигналы от внешних источников, аудит логов.
2. **Действия:**
   a. Изоляция скомпрометированного сегмента сети.

   b. Отзыв доступов подозрительных учетных записей.
   c. Уведомление регулятора (Роскомнадзор) в соответствии с ФЗ-152 (в течение 24 часов).
3. **Восстановление:** установка патча уязвимости, сброс сессий пользователей, расследование инцидента.

Рассинхронизация остатков (RSK01_05, RSK02_06)

1. **Обнаружение:** Жалобы пользователей, алерт расхождения кэша и БД.
2. **Действия:**
   a. Принудительная очистка кэша товаров.
   b. Запуск процесса пересчета остатков.
   c. Временная маркировка объявлений «Проверка наличия» для новых заказов.
3. **Восстановление:** Сверка с сервисом склада/продавца.

3.11.5. Коммуникация

- **Внутренняя:** Чат инцидентов, баг-трекер.
- **Внешняя:**
  - Страница состояния системы.
  - Рассылка пользователям при влиянии на их заказы (RSK02_07).
  - Официальное заявление при утечке данных (RSK02_08).

3.11.6. Пост-инцидентный анализ

По каждому инциденту уровня "критический" или "высокий" в течение 2 рабочих дней проводится совещание и формируется отчет:

- Хронология событий.
- Корневая причина.
- План предотвращающих действий на будущее с ответственными и сроками.
- Обновление документации и тестов на основе выявленных проблем.

3.12. Схема взаимодействия

3.12.1. Принципы взаимодействия

Тип	Когда используется	Протокол
Синхронный	Мгновенный ответ, строгая консистентность	HTTPS (REST), gRPC
Асинхронный	Устойчивость к сбоям, fan-out, eventual consistency	Kafka

Тип	Когда используется	Протокол
Webhook	Внешний провайдер инициирует событие	HTTPS

3.12.2. Синхронные взаимодействия (прямые вызовы сервис→сервис)

Компонент A	Компонент B	Протокол
Client	GeoDNS	DNS
GeoDNS	API Gateway	HTTPS
Client	CDN	HTTPS
API Gateway	Auth Service	gRPC
API Gateway	User Service	HTTP
API Gateway	Ticket Service	HTTP
API Gateway	Search Service	HTTP
API Gateway	Order Service	HTTP
API Gateway	Payment Service	HTTP
API Gateway	Redis	RESP
Order Service	Ticket Service	gRPC
Order Service	Payment Service	gRPC
Order Service	Delivery Service	gRPC
Billing Service	Payment Service	gRPC

3.12.3. Асинхронные взаимодействия (через Kafka)

Producers Kafka

Producer	Topic	События
Ticket Service	ticket.events	ticket.created, ticket.updated, ticket.published, ticket.deleted, ticket.reserved
Ticket Service	message.events	message.created, message.sent, message.delivered, message.read
Order Service	order.events	order.created, order.confirmed, order.cancelled, order.completed, order.shipped
Payment Service	payment.events	payment.created, payment.processing, payment.succeeded, payment.failed, payment.refunded
Delivery Service	delivery.events	delivery.created, delivery.in_transit, delivery.completed, delivery.failed

Consumers Kafka

Consumer	Topic	Действие
Search Service	ticket.events	Индексация объявлений в ElasticSearch
Order Service	payment.events	Обновление статуса заказа после оплаты
Order Service	delivery.events	Обновление статуса заказа по доставке
Billing Service	order.events	Расчёт комиссии и выплата продавцу после order.completed
Analytics Service (ClickHouse)	ticket.events	Сбор аналитики по объявлениям
Analytics Service (ClickHouse)	order.events	Сбор аналитики по заказам
Analytics Service (ClickHouse)	payment.events	Сбор аналитики по платежам
Analytics Service (ClickHouse)	delivery.events	Сбор аналитики по доставке
Analytics Service (ClickHouse)	message.events	Сбор аналитики по сообщениям

Полная схема Kafka (Producer → Topic → Consumer)

Producer	Topic	Consumer
Ticket Service	ticket.events	Search Service
Ticket Service	ticket.events	Analytics Service (ClickHouse)
Ticket Service	message.events	Analytics Service (ClickHouse)
Order Service	order.events	Billing Service
Order Service	order.events	Analytics Service (ClickHouse)
Payment Service	payment.events	Order Service
Payment Service	payment.events	Analytics Service (ClickHouse)
Delivery Service	delivery.events	Order Service
Delivery Service	delivery.events	Analytics Service (ClickHouse)

Topics и партиционирование

Topic	Ключ партиционирования	Retention
ticket.events	ticket_id	7 дней
message.events	conversation_id	7 дней
order.events	order_id	7 дней
payment.events	payment_id	30 дней
delivery.events	delivery_id	7 дней

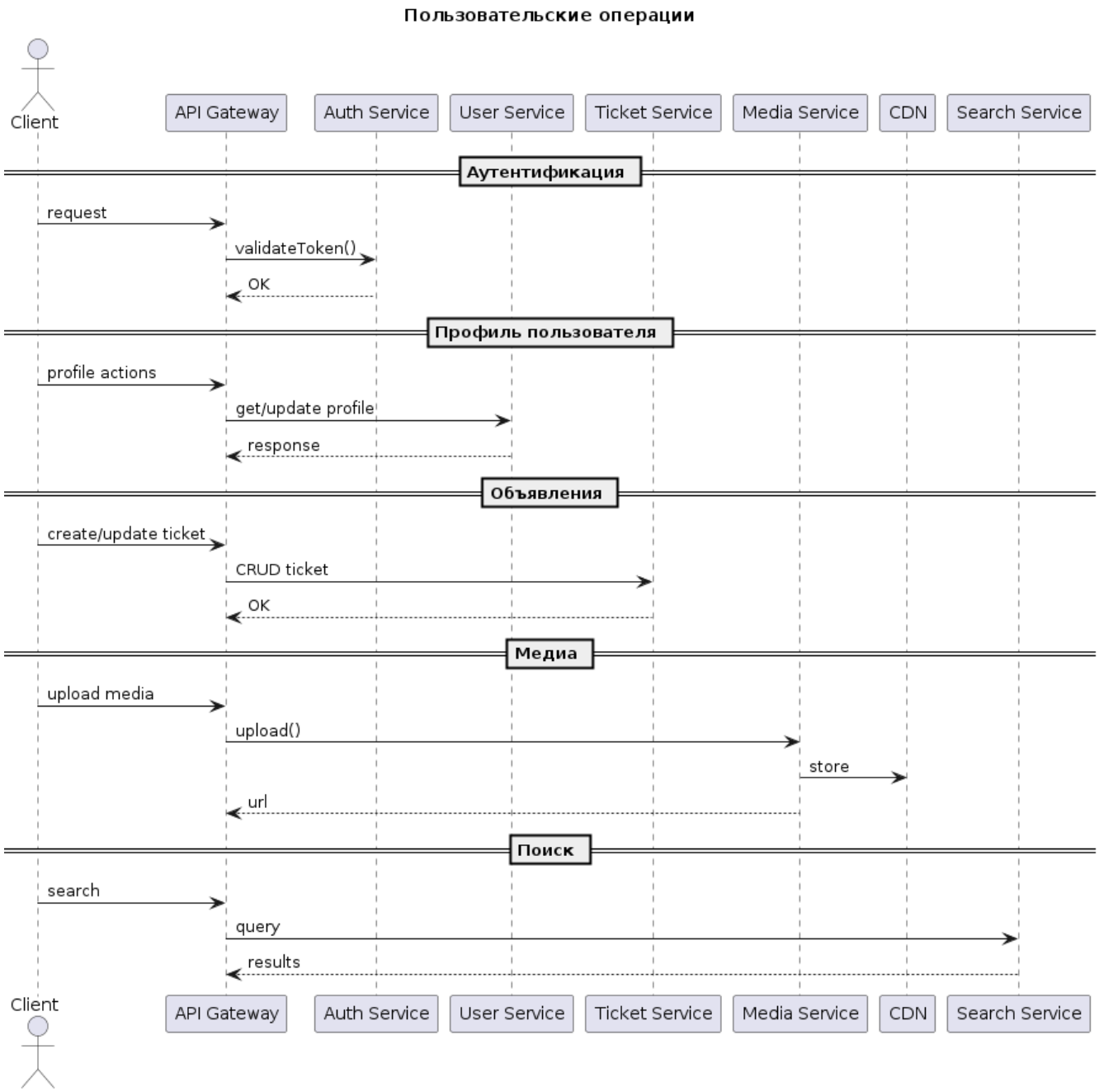
Consumer группы

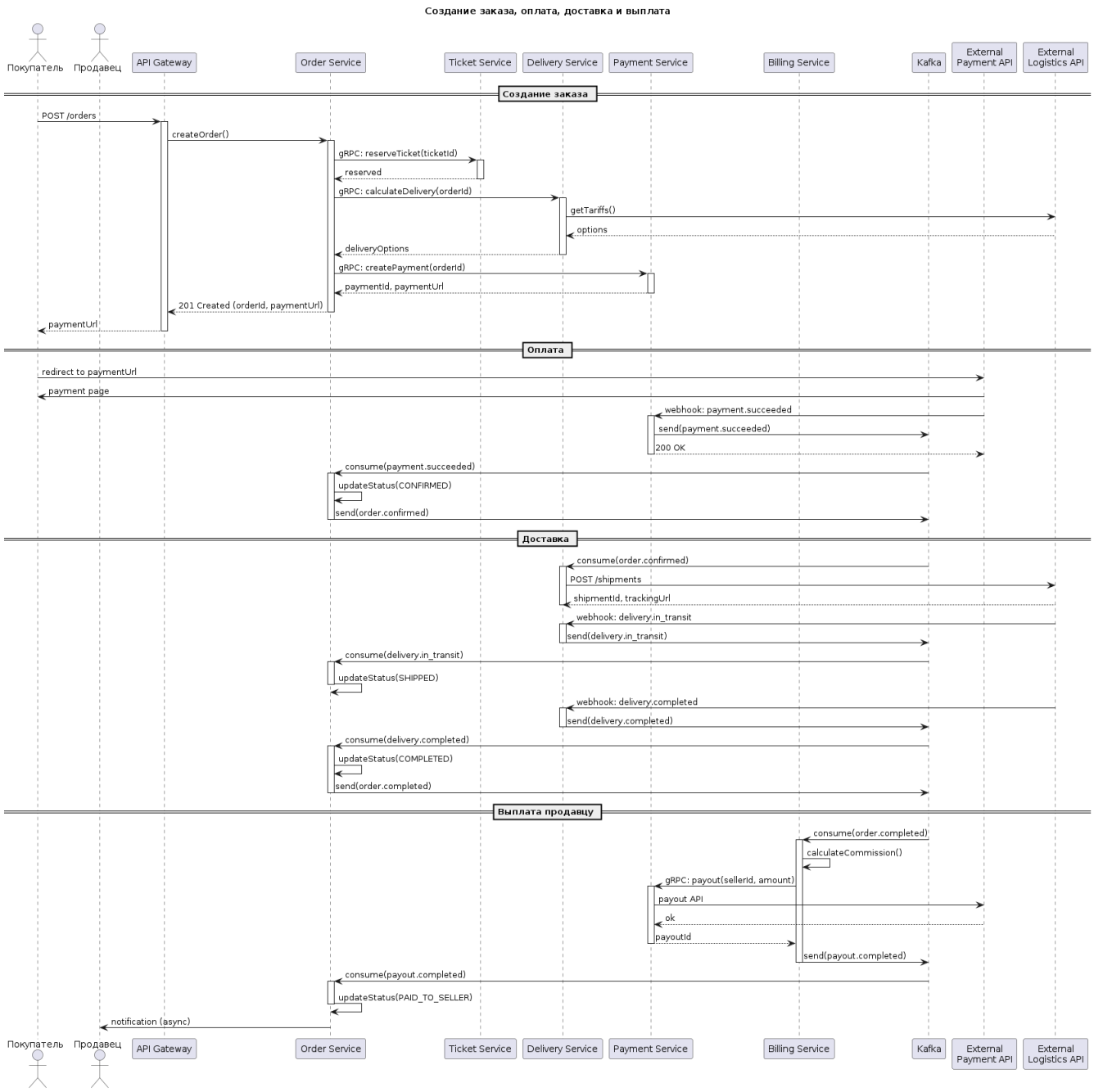
Consumer группа	Топики
search-service-group	ticket.events
order-service-group	payment.events, delivery.events
billing-service-group	order.events
analytics-service-group	ticket.events, order.events, payment.events, delivery.events, message.events

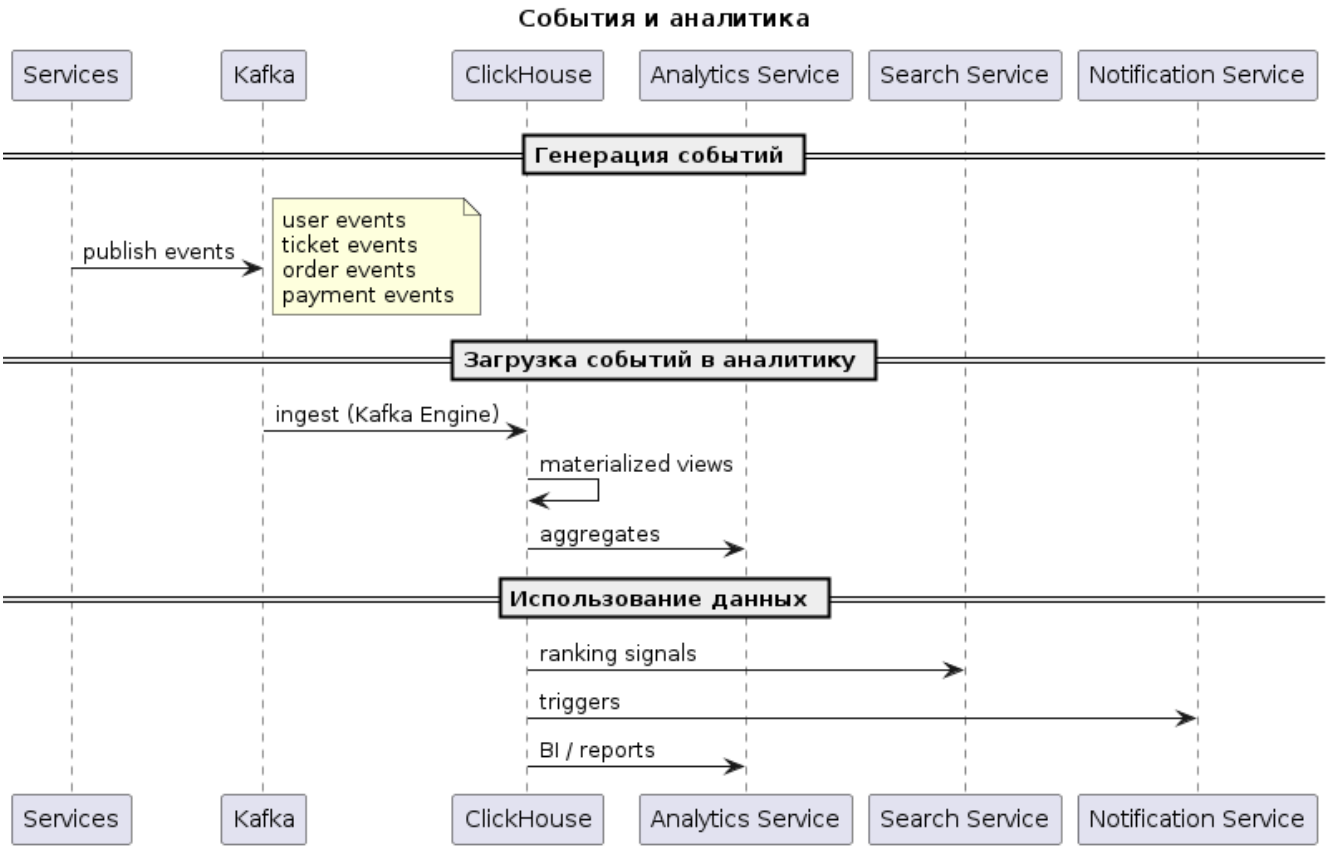
3.12.4. Внешние взаимодействия (Webhook)

Компонент A	Компонент B	Протокол
Платёжный агрегатор	Payment Service	HTTPS (webhook)
Логистический API	Delivery Service	HTTPS (webhook)

3.12.5. Схемы процессов


Пользовательские операции
Client
API Gateway
Auth Service
User Service
Ticket Service
Media Service
CDN
Search Service
Аутентификация
request
validateToken()
OK
Профиль пользователя
profile actions
get/update profile
response
Объявления
create/update ticket
CRUD ticket
OK
Медиа
upload media
upload()
store
url
Поиск
search
query
results


Создание заказа, оплата, доставка и выплата
Покупатель
Продавец
API Gateway
Order Service
Ticket Service
Delivery Service
Payment Service
Billing Service
Kafka
External Payment API
External Logistics API
Создание заказа
POST /orders
createOrder()
gRPC: reserveTicket(ticketId)
reserved
gRPC: calculateDelivery(orderId)
getTariffs()
options
deliveryOptions
gRPC: createPayment(orderId)
paymentId, paymentUrl
201 Created (orderId, paymentUrl)
paymentUrl
Оплата
redirect to paymentUrl
payment page
webhook: payment.succeeded
send(payment.succeeded)
200 OK
consume(payment.succeeded)
updateStatus(CONFIRMED)
send(order.confirmed)
Доставка
consume(order.confirmed)
POST /shipments
shipmentId, trackingUrl
webhook: delivery.in_transit
send(delivery.in_transit)
consume(delivery.in_transit)
updateStatus(SHIPPED)
webhook: delivery.completed
send(delivery.completed)
consume(delivery.completed)
updateStatus(COMPLETED)
send(order.completed)
Выплата продавцу
consume(order.completed)
calculateCommission()
gRPC: payout(sellerId, amount)
payout API
ok
payoutId
send(payout.completed)
consume(payout.completed)
updateStatus(PAID_TO_SELLER)
notification (async)



3.13. Форматы взаимодействия

Раздел описывает структуру и формат данных при взаимодействии компонентов. Все форматы соответствуют схемам данных, описанным в разделе «Схемы данных». Для всех операций, не имеющих явного примера, используются те же принципы и соответствующие схемы из раздела «Схемы данных».

3.13.1. DNS / GeoDNS

Формат	Описание
A-запись (Address Record)	Преобразует доменное имя в IPv4-адрес. Используется для георасределения трафика между ЦОД (Москва/Краснодар). GeoDNS возвращает IP-адрес ближайшего ЦОД на основе местоположения клиента.
SRV-запись (Service Record)	Определяет хост и порт для конкретного сервиса. Используется для определения активного API Gateway в каждом ЦОД. Включает приоритет и вес для failover.

3.13.2. HTTP/REST (JSON)

Формат для внешних API (Client → API Gateway).

Общие принципы

- Кодировка: UTF-8
- Content-Type: application/json
- Версионирование: через URL path (`/v1/`, `/v2/`)
- Пагинация: cursor-based (after_id, limit) — клиент передаёт `limit` и идентификатор последнего полученного элемента
- Ошибки: формат RFC 9457 (Problem Details for HTTP APIs)

Форматы запросов и ответов

Все форматы запросов и ответов соответствуют схемам из раздела «Схемы данных»:

- `User`, `SellerProfile` — для операций с пользователями
- `Ticket`, `Category` — для операций с объявлениями
- `Order`, `Reservation` — для операций с заказами
- `Message`, `Conversation` — для операций с сообщениями
- `Payment`, `Payout`, `Refund`, `Transaction` — для платёжных операций
- `Shipment`, `DeliveryService` — для операций доставки
- `Review`, `Notification` — для отзывов и уведомлений

Поля запросов и ответов соответствуют полям этих схем (см. раздел «Схемы данных»).

Пример: создание объявления (POST /v1/tickets)

Запрос (соответствует схемам `Ticket`, `Category`):

```
{
    "title": "string",                  // Ticket.title
    "description": "string",            // Ticket.description
    "category_id": "uuid",              // Ticket.category_id
    "price": 0,                         // Ticket.price
    "total_quantity": 0,                // Ticket.total_quantity
    "images": ["string"]                // массив URL изображений
}
```

Ответ (201 Created):

```
{
    "ticket_id": "uuid",                // Ticket.ticket_id
    "status": "string",                 // Ticket.status
    "created_at": "datetime"            // Ticket.created_at
}
```

Пример: создание заказа (POST /v1/orders)

Запрос (соответствует схемам `Order`, `Reservation`):

```
{
    "ticket_id": "uuid",                // Order.ticket_id
    "quantity": 0,                      // Order.quantity
    "delivery_address": {               // адрес доставки
        "city": "string",
        "street": "string",
        "house": "string"
    }
}
```

Ответ (201 Created):

```
{
    "order_id": "uuid",                 // Order.order_id
    "status": "string",                 // Order.status
    "total_price": 0,                   // Order.total_price
    "payment_url": "string",            // URL для оплаты
    "created_at": "datetime"            // Order.created_at
}
```

Пример ответа с ошибкой (4xx/5xx)

Формат RFC 9457:

```
{
    "type": "string",                   // URI типа ошибки
    "title": "string",                  // краткое описание
    "status": 0,                        // HTTP статус
    "detail": "string",                 // детальное описание
    "instance": "string"                // URI запроса
}
```

Остальные операции

Для остальных операций (просмотр профиля, редактирование объявления, поиск, сообщения, получение статуса заказа и т.д.) используются:

- Те же принципы кодирования и версионирования
- Соответствующие схемы из раздела «Схемы данных»
- Поля, указанные в схемах (например, `GET /v1/users/{user_id}` возвращает `User`, `GET /v1/tickets/{ticket_id}` возвращает `Ticket` и т.д.)

3.13.3. gRPC (Protobuf)

Формат для внутренних синхронных вызовов между сервисами внутри ЦОД.

Все gRPC-сообщения соответствуют схемам данных:

- `User`, `UserRole`, `Role` — для Auth Service
- `Order`, `Payment` — для Payment Service
- `Order`, `Shipment`, `DeliveryService` — для Delivery Service
- `Payout`, `Transaction` — для Billing Service

Auth Service (auth.proto)

```
syntax = "proto3";

service AuthService {
    rpc ValidateToken (ValidateTokenRequest) returns (ValidateTokenResponse);
    rpc Authenticate (AuthenticateRequest) returns (AuthenticateResponse);
}

message ValidateTokenRequest {
    string token = 1;                    // JWT или session token
}

message ValidateTokenResponse {
    bool valid = 1;                      // признак валидности токена
    string user_id = 2;                  // User.user_id
    repeated string roles = 3;           // роли из UserRole + Role
    int64 expires_at = 4;                // timestamp истечения
}

message AuthenticateRequest {
    string login = 1;                    // email или телефон (User.email/phone)
    string password = 2;                 // пароль (User.password_hash)
}

message AuthenticateResponse {
    string session_id = 1;               // идентификатор сессии
    string jwt_token = 2;                // JWT токен
    int64 expires_in = 3;                // время жизни в секундах
}
```

Payment Service (payment.proto)

```
syntax = "proto3";

service PaymentService {
    rpc CreatePayment (CreatePaymentRequest) returns (CreatePaymentResponse);
```

```
    rpc Payout (PayoutRequest) returns (PayoutResponse);
}

message CreatePaymentRequest {
    string order_id = 1;                // Order.order_id
    int64 amount = 2;                   // сумма в копейках (Order.total_price)
    string currency = 3;                // валюта (RUB)
    string return_url = 4;              // URL возврата после оплаты
    string webhook_url = 5;             // URL для уведомлений
}

message CreatePaymentResponse {
    string payment_id = 1;              // Payment.payment_id
    string payment_url = 2;             // URL для редиректа на оплату
    string status = 3;                  // Payment.status
}

message PayoutRequest {
    string seller_id = 1;               // SellerProfile.seller_id
    int64 amount = 2;                   // сумма выплаты
    string currency = 3;                // валюта
    string payout_id = 4;               // Payout.payout_id
}

message PayoutResponse {
    string payout_id = 1;               // Payout.payout_id
    string status = 2;                  // Payout.status
    string transaction_id = 3;          // Transaction.transaction_id
}
```

Delivery Service (delivery.proto)

```
syntax = "proto3";

service DeliveryService {
    rpc CalculateDelivery (DeliveryRequest) returns (DeliveryResponse);
    rpc CreateShipment (ShipmentRequest) returns (ShipmentResponse);
}

message DeliveryRequest {
    string order_id = 1;                // Order.order_id
    string address = 2;                 // адрес доставки
    int32 quantity = 3;                 // количество товаров (Order.quantity)
}

message DeliveryResponse {
    repeated DeliveryOption options = 1; // варианты доставки
}

message DeliveryOption {
```

```
    string delivery_service_id = 1;      // DeliveryService.delivery_service_id
    string name = 2;                     // DeliveryService.name
    int64 price = 3;                     // стоимость доставки
    int32 days = 4;                      // срок в днях
}

message ShipmentRequest {
    string order_id = 1;                 // Order.order_id
    string delivery_service_id = 2;      // DeliveryService.delivery_service_id
    string address = 3;                  // адрес доставки
}

message ShipmentResponse {
    string shipment_id = 1;              // Shipment.shipment_id
    string tracking_number = 2;          // Shipment.tracking_number
    string status = 3;                   // Shipment.status
}
```

3.13.4. HTTP Webhook (JSON)

Формат для внешних асинхронных колбэков.

Все webhook-сообщения соответствуют схемам `Payment`, `Transaction`, `Shipment`.

Webhook платежей

```
POST /webhook/payment
{
    "payment_id": "uuid",                // Payment.payment_id
    "order_id": "uuid",                  // Order.order_id
    "status": "string",                  // Payment.status (Succeeded, Failed)
    "provider": "string",                // Payment.provider
    "provider_txn_id": "string",         // Transaction.external_reference
    "amount": 0,                         // Payment.amount
    "currency": "string",                // валюта
    "timestamp": "datetime",             // время события
    "signature": "string"                // подпись для верификации
}
```

Webhook доставки

```
POST /webhook/delivery
{
    "shipment_id": "uuid",               // Shipment.shipment_id
    "order_id": "uuid",                  // Order.order_id
    "status": "string",                  // Shipment.status
    "tracking_number": "string",         // Shipment.tracking_number
    "carrier": "string",                 // DeliveryService.name
```

```
    "timestamp": "datetime"             // время события
}
```

3.13.5. Kafka / Event bus

Формат для асинхронных событий.

Все события соответствуют схемам `Ticket`, `Order`, `Payment`, `Shipment`, `Message`.

Схема события ticket.events

```
{
    "event_id": "uuid",                 // уникальный идентификатор события
    "event_type": "string",             // created, updated, published, deleted,
reserved
    "ticket_id": "uuid",                // Ticket.ticket_id
    "seller_id": "uuid",                // Ticket.seller_id
    "data": "json",                     // изменённые поля Ticket
    "timestamp": "datetime"             // время события
}
```

Схема события order.events

```
{
    "event_id": "uuid",                 // уникальный идентификатор события
    "event_type": "string",             // created, confirmed, cancelled, completed,
shipped
    "order_id": "uuid",                 // Order.order_id
    "buyer_id": "uuid",                 // Order.buyer_id
    "seller_id": "uuid",                // Order.seller_id
    "total_price": 0,                   // Order.total_price
    "status": "string",                 // Order.status
    "timestamp": "datetime"             // время события
}
```

Схема события payment.events

```
{
    "event_id": "uuid",                 // уникальный идентификатор события
    "event_type": "string",             // created, processing, succeeded, failed,
refunded
    "payment_id": "uuid",               // Payment.payment_id
    "order_id": "uuid",                 // Order.order_id
    "amount": 0,                        // Payment.amount
    "status": "string",                 // Payment.status
    "timestamp": "datetime"             // время события
```

```
}
```

Схема события delivery.events

```
{
    "event_id": "uuid",                 // уникальный идентификатор события
    "event_type": "string",             // created, in_transit, completed, failed
    "shipment_id": "uuid",              // Shipment.shipment_id
    "order_id": "uuid",                 // Order.order_id
    "status": "string",                 // Shipment.status
    "tracking_number": "string",        // Shipment.tracking_number
    "timestamp": "datetime"             // время события
}
```

Схема события message.events

```
{
    "event_id": "uuid",                 // уникальный идентификатор события
    "event_type": "string",             // created, sent, delivered, read
    "message_id": "uuid",               // Message.message_id
    "conversation_id": "uuid",          // Message.conversation_id
    "sender_id": "uuid",                // Message.sender_id
    "body": "string",                   // Message.body (обрезанный для аналитики)
    "status": "string",                 // Message.status
    "timestamp": "datetime"             // время события
}
```

Ключи партиционирования Kafka

Topic	Ключ
ticket.events	ticket_id
order.events	order_id
payment.events	payment_id
delivery.events	shipment_id
message.events	conversation_id

3.13.6. Redis / Cache (RESP)

Формат для кэширования.

Шаблон ключа	TTL	Значение (JSON)
`session:{session_id}`	24h	`{"user_id": "uuid", "roles": ["string"], "expires_at": "datetime"}`
`token:{jwt}`	1h	`{"user_id": "uuid", "blocked": false}`

Шаблон ключа	TTL	Значение (JSON)
`search:{query_hash}`	5m	`{"ticket_ids": ["uuid"], "total": 0}`
`ticket:{ticket_id}`	10m	полный объект Ticket (см. схему)
`user:{user_id}`	10m	полный объект User (см. схему)

3.13.7. S3 / CDN

Формат для медиафайлов.

Схема URL

```
https://cdn.{city}.example.com/media/{ticket_id}/{file_id}/{version}.{ext}
```

где: - `{city}` — spb или kzn (ближайший CDN узел) - `{ticket_id}` — Ticket.ticket_id - `{file_id}` — уникальный идентификатор файла - `{version}` — версия файла (для инвалидации кэша) - `{ext}` — расширение файла

Метаданные (S3)

```
{
    "ticket_id": "uuid",                // Ticket.ticket_id
    "user_id": "uuid",                  // Ticket.seller_id
    "content_type": "string",           // MIME тип
    "size_bytes": 0,                    // размер в байтах
    "width": 0,                         // ширина изображения
    "height": 0,                        // высота изображения
    "created_at": "datetime"            // время загрузки
}
```

3.14. Система хранения данных

3.14.1. Общая архитектура

Система хранения данных разделена по бизнес-доменам и типам нагрузки. Используется комбинация реляционных и нереляционных хранилищ, кэширования и аналитического хранилища для масштабируемости и разделения OLTP/OLAP нагрузки.

3.14.2. Хранилища по сервисам

Компонент	Тип хранилища	Георасприделение	Репликация / Шардинг	Особенности
Сервис авторизации	PostgreSQL + Redis	Москва / Краснодар	PostgreSQL: master-replica; Redis: master-replica в каждом ЦОД	credentials (PostgreSQL), sessions + tokens (Redis, TTL)
Сервис пользователей	PostgreSQL	Москва / Краснодар	Geo-sharding по user_id	профильные данные пользователей
Сервис объявлений	PostgreSQL	Москва / Краснодар	Geo-sharding по ticket_id	объявления + модерация + сообщения
Сервис поиска / рекомендаций	Elasticsearch	Москва / Краснодар	shards + replicas	полнотекстовый поиск и ранжирование
Сервис заказов	PostgreSQL	Москва / Краснодар	Geo-sharding по order_id	транзакционные данные заказов
Платёжный сервис	PostgreSQL	Москва / Краснодар	master + replicas	финансовые операции
Сервис расчётов	PostgreSQL	Москва	single-writer + replica	балансы и выплаты
Сервис доставки	PostgreSQL	Москва / Краснодар	geo-sharding по order_id	логистика и статусы доставки
Сервис медиа	S3 + CDN	Москва / Краснодар + СПб / Казань (edge)	репликация объектов + edge caching	хранение и раздача медиа
Кэш	Redis	Москва / Краснодар	cluster (partitioning + replicas)	сессии, токены, кэш данных
Аналитический сервис	ClickHouse	Москва / Краснодар	distributed + replicated tables	агрегации событий, BI, метрики, рекомендации, нотификации
Асинхронные события	Kafka	Москва / Краснодар	partitioning + replication	event bus системы
Мониторинг / логирование	ELK	Москва / Краснодар	отдельные кластеры по ЦОД	централизованный сбор логов

3.14.3. Репликация и шардирование

- PostgreSQL
  - geo-sharding по user_id / ticket_id / order_id
  - master-replica или single-writer по домену
  - read-replicas для масштабирования
- Elasticsearch
  - shard + replica модель
  - синхронизация через Kafka
- Redis
  - cluster mode
  - TTL для сессий и токенов
- S3 / CDN
  - репликация между регионами
  - edge-кэширование (СПб, Казань)
- Kafka
  - partitioning по entity_id
  - replication между ЦОД
- ClickHouse (аналитика)
  - ReplicatedMergeTree
  - Distributed tables между Москвой и Краснодаром
  - ingestion через Kafka Engine
  - materialized views для агрегаций

3.14.4. CQRS и Event-driven

- Сервис объявлений:
  - write: PostgreSQL
  - read: Elasticsearch
  - события публикуются в Kafka
- Event-driven слой:
  - все сервисы публикуют события в Kafka
  - ClickHouse потребляет события для аналитики
  - аналитический сервис формирует агрегаты и метрики

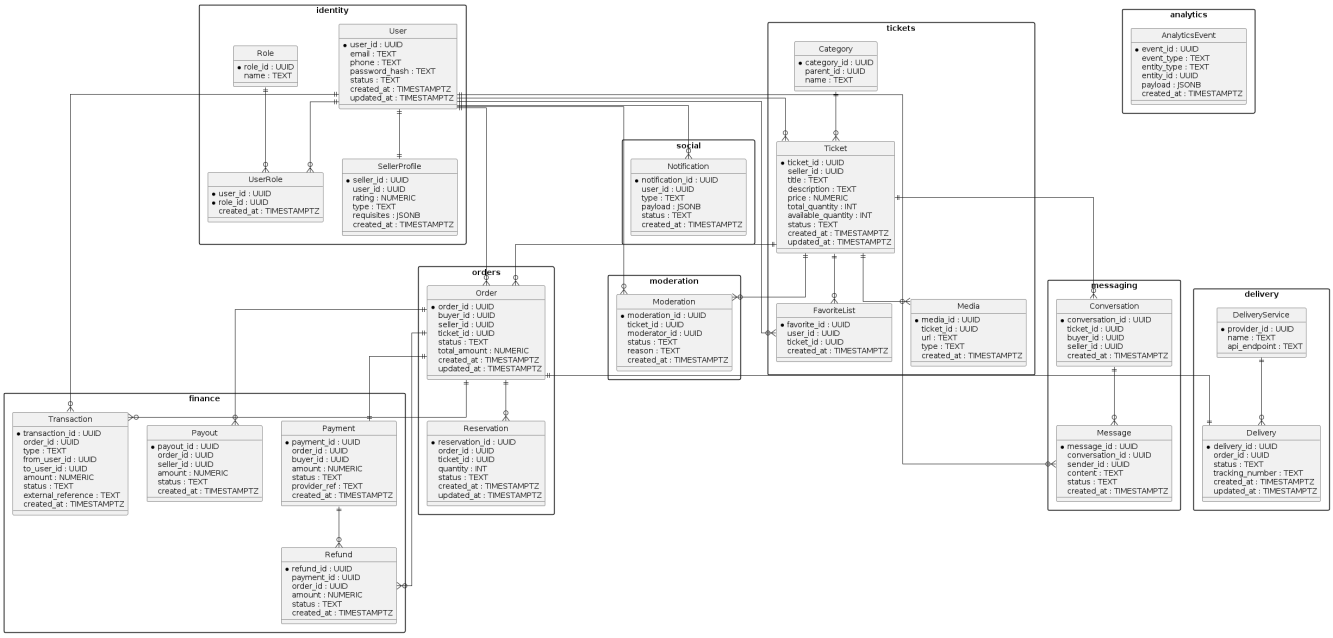
3.14.5. Итоговая модель хранения данных

Тип данных	Хранилище	Георасределение	Назначение
Пользователи / аутентификация	PostgreSQL + Redis	Москва / Краснодар	профили, сессии, токены
Объявления	PostgreSQL	Москва / Краснодар	объявления, модерация, сообщения
Поиск / рекомендации	Elasticsearch	Москва / Краснодар	поиск и ранжирование
Заказы / платежи / расчёты	PostgreSQL	Москва / Краснодар	транзакции и финансы
Доставка	PostgreSQL	Москва / Краснодар	логистика и статусы
Медиа	S3 + CDN	Москва / Краснодар + edge	хранение и раздача контента
Кэш	Redis	Москва / Краснодар	временные данные и сессии
Аналитика	ClickHouse	Москва / Краснодар	события, агрегации, BI

etcd-кластер из трёх узлов: - Москва: leader - Краснодар: follower - Казань: witness (участвует в кворуме, не хранит данные)

При отказе Москвы кворум (Краснодар + Казань) выбирает новый мастер для платежей и расчётов.

3.15. Схемы данных



3.15.1. Пользователи и роли

User

Поле	Тип	Описание
user_id	UUID PK	Идентификатор пользователя
email	TEXT	Электронная почта
phone	TEXT	Телефон
password_hash	TEXT	Хэш пароля
status	TEXT	Статус пользователя

Role

Поле	Тип	Описание
role_id	UUID PK	Идентификатор роли
name	TEXT	Название роли (buyer, seller, moderator…)

UserRole [связь пользователя с ролью]

Поле	Тип	Описание
user_id	UUID FK	Ссылка на User
role_id	UUID FK	Ссылка на Role

SellerProfile [профиль продавца]

Поле	Тип	Описание
seller_id	UUID PK	Идентификатор профиля продавца
user_id	UUID FK	Ссылка на User
type	TEXT	Тип продавца (физическое/юр. лицо)
rating	NUMERIC	Рейтинг продавца
legal_info	JSONB	Юридические реквизиты

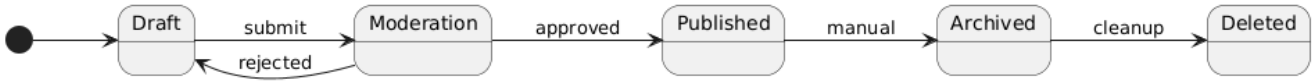
3.15.2. Объявления и сообщения

Ticket [объявление]

Поле	Тип	Описание
ticket_id	UUID PK	Идентификатор объявления
seller_id	UUID FK	Ссылка на User (seller)
title	TEXT	Заголовок

Поле	Тип	Описание
description	TEXT	Описание
category_id	UUID FK	Ссылка на Category
price	NUMERIC	Цена
total_quantity	INT	Всего единиц
reserved_quantity	INT	Зарезервировано
available_quantity	INT	Доступно
status	TEXT	Статус (Draft, Published…)
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

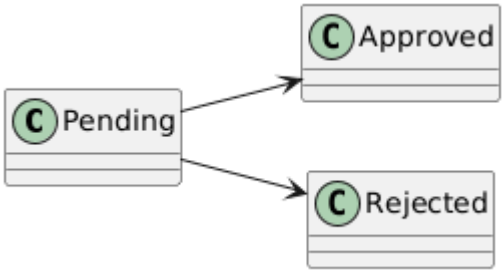
Состояния объявления



Moderation [Резолюция модератора]

Поле	Тип	Описание
moderation_id	UUID PK	Идентификатор модерации
ticket_id	UUID FK	Ссылка на Ticket
status	TEXT	Статус (Pending, Approved, Rejected)
moderator_id	UUID FK	Ссылка на User (moderator)
comment	TEXT	Комментарий модератора
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

Состояния объявления по модерации



Conversation [диалог]

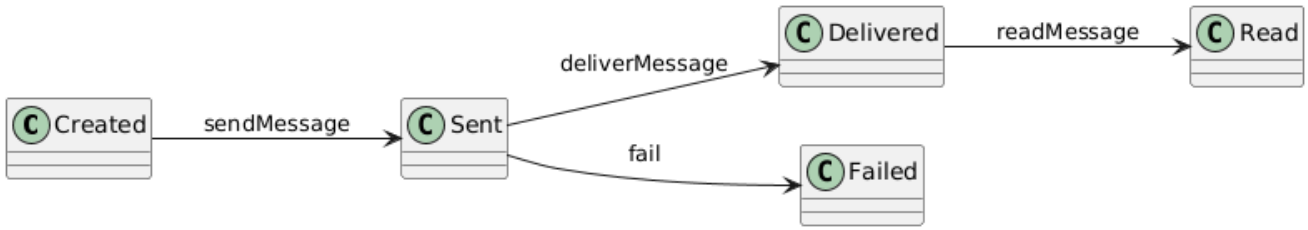
Поле	Тип	Описание
conversation_id	UUID PK	Идентификатор диалога

Поле	Тип	Описание
ticket_id	UUID FK	Ссылка на Ticket
buyer_id	UUID FK	Ссылка на User
seller_id	UUID FK	Ссылка на User

Message [Сообщение диалога]

Поле	Тип	Описание
message_id	UUID PK	Идентификатор сообщения
conversation_id	UUID FK	Ссылка на Conversation
sender_id	UUID FK	Отправитель (User)
body	TEXT	Текст сообщения
status	TEXT	Статус (Created, Sent, Read)
created_at	timestamp	время создания (зональное)
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

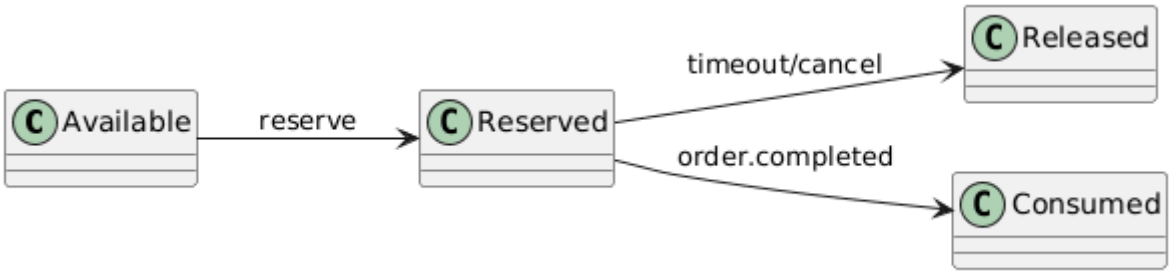
Состояния сообщений



Reservation [Резерв товара]

Поле	Тип	Описание
reservation_id	UUID PK	Идентификатор резерва
ticket_id	UUID FK	Ссылка на Ticket
order_id	UUID FK	Ссылка на Order
quantity	INT	Количество зарезервированное
status	TEXT	Статус (Available, Reserved...)
expires_at	TIMESTAMP	Время истечения резерва
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

Состояния товара при резервировании



Category [Категория товара (классификция по каталогу)]

Поле	Тип	Описание
category_id	UUID PK	Идентификатор категории
parent_id	UUID FK	Родительская категория
name	TEXT	Название категории

FavoriteList [wishlist пользователя]

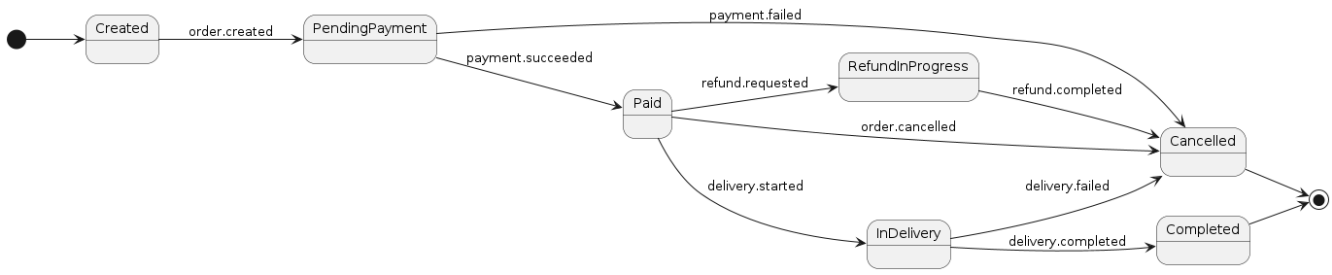
Поле	Тип	Описание
user_id	UUID FK	Ссылка на User
ticket_id	UUID FK	Ссылка на Ticket

3.15.3. Заказы и платежи

Order [Заказ]

Поле	Тип	Описание
order_id	UUID PK	Идентификатор заказа
buyer_id	UUID FK	Ссылка на User
seller_id	UUID FK	Ссылка на User
ticket_id	UUID FK	Ссылка на Ticket
quantity	INT	Количество
total_price	NUMERIC	Сумма заказа
status	TEXT	Статус заказа (Created, PendingPayment…)
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

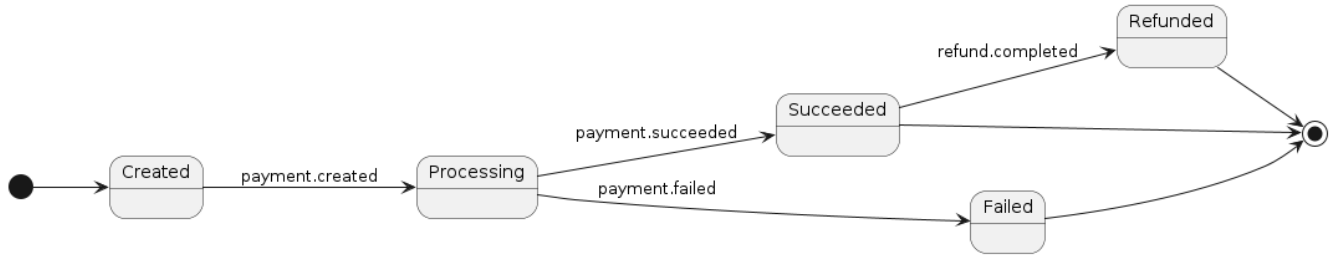
Состояния заказа



Payment [Оплата заказа]

Поле	Тип	Описание
payment_id	UUID PK	Идентификатор платежа
order_id	UUID FK	Ссылка на Order
amount	NUMERIC	Сумма платежа
status	TEXT	Статус (Processing, Succeeded…)
provider	TEXT	Платежный провайдер
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

Состояния оплаты

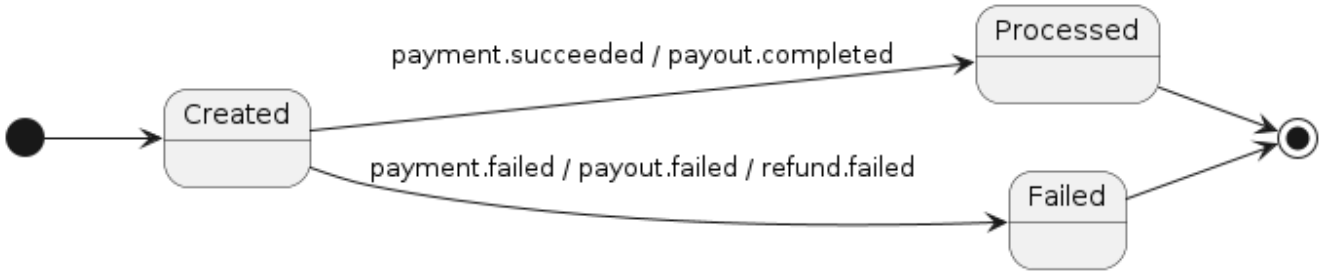


Transaction [Транзакция]

Поле	Тип	Описание
transaction_id	UUID PK	Уникальный идентификатор операции
order_id	UUID FK	Связь с заказом (контекст финансовой операции)
type	TEXT	Тип операции: PAYMENT / REFUND / PAYOUT
from_user_id	UUID	Инициатор списания (плательщик / система)
to_user_id	UUID	Получатель (продавец / покупатель / система)
amount	NUMERIC	Сумма операции
currency	TEXT	Валюта (RUB)
status	TEXT	CREATED / PROCESSED / FAILED

Поле	Тип	Описание
external_reference	TEXT	ID операции у платёжного провайдера (если есть)
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное). *Note: для транзакций ни к чему, но пусть будет*

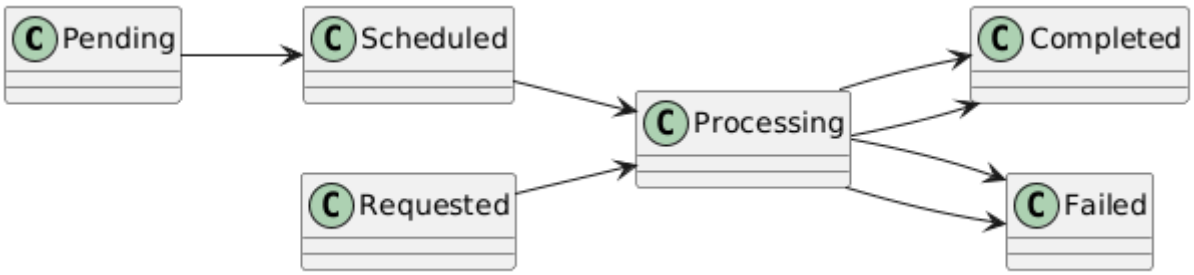
Состояния транзакций



Payout [Выплата]

Поле	Тип	Описание
payout_id	UUID PK	Идентификатор выплаты
seller_id	UUID FK	Ссылка на SellerProfile
amount	NUMERIC	Сумма выплаты
status	TEXT	Статус
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

Состояния выплат

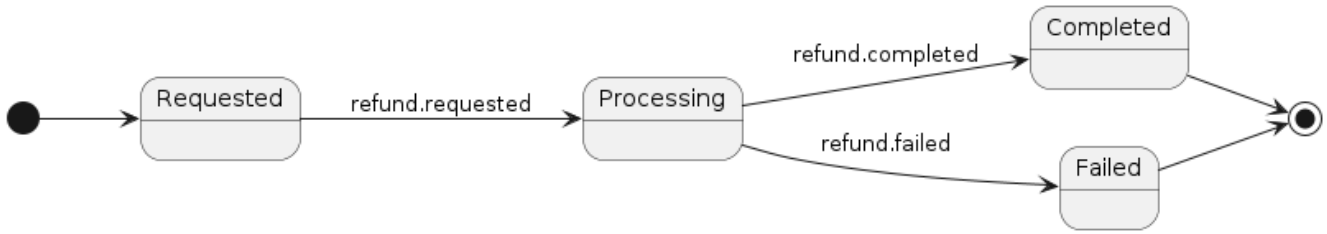


Refund [Возврат]

Поле	Тип	Описание
refund_id	UUID PK	Идентификатор возврата
payment_id	UUID FK	Ссылка на Payment
amount	NUMERIC	Сумма возврата

Поле	Тип	Описание
status	TEXT	Статус
created_at	timestamp	время создания (зональное)
updated_at	timestamp	время изменения (зональное)

Состояния возвратов

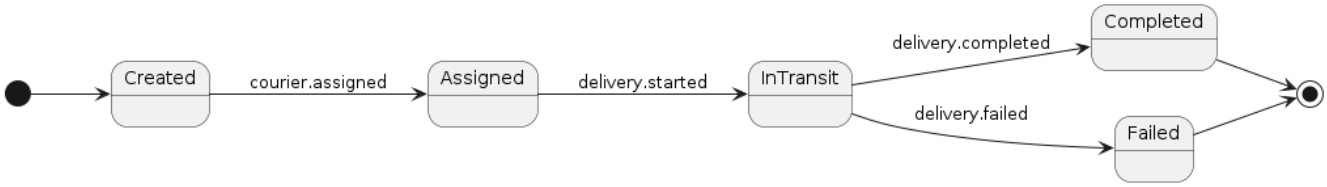


3.15.4. Доставка

Shipment [собственно доставка]

Поле	Тип	Описание
shipment_id	UUID PK	Идентификатор доставки
order_id	UUID FK	Ссылка на Order
delivery_service_id	UUID FK	Ссылка на DeliveryService
tracking_number	TEXT	Трек-номер
status	TEXT	Статус доставки

Состояния доставки



DeliveryService [логистическая служба]

Поле	Тип	Описание
delivery_service_id	UUID PK	Идентификатор службы доставки
name	TEXT	Название службы
api_endpoint	TEXT	API службы

3.15.5. Поддержка

Notification [Уведомление]

Поле	Тип	Описание
notification_id	UUID PK	Идентификатор уведомления
user_id	UUID FK	Ссылка на User
type	TEXT	Тип уведомления
payload	JSONB	Данные уведомления
status	TEXT	Статус

Review [Отзыв]

Поле	Тип	Описание
review_id	UUID PK	Идентификатор отзыва
author_id	UUID FK	Автор (User)
target_id	UUID FK	Получатель отзыва (User)
rating	INT	Оценка
text	TEXT	Текст отзыва

Notification [Извещение]

Поле	Тип	Описание
notification_id	UUID	Уникальный идентификатор уведомления (PK)
user_id	UUID	Получатель уведомления (FK → User)
type	TEXT	Тип уведомления: order / payment / delivery / message / moderation / system
payload	JSONB	Контекст события для UI и обработки
status	TEXT	created / sent / read / failed
created_at	timestamp	Время создания уведомления

Глава 4. Безопасность

4.1. Схема защиты и управления доступом

4.1.1. Общие принципы

- Минимально необходимые права
- Разделение обязанностей
- Проверка владения ресурсом
- Усиленная аутентификация для критичных операций
- Централизованная точка входа (API Gateway)

4.1.2. Ролевая модель (RBAC)

Контроль доступа основан на ролях пользователей.

Матрица доступа

Операция / Роль	Гость	Покупатель	Продавец	Модератор	Аналитик	Администратор
Просмотр объявлений	✔	✔	✔	✔	✔	✔
Управление объявлениями	-	-	✔ (свои)	✔	-	✔
Модерация	-	-	-	✔	-	✔
Создание заказа	-	✔	-	-	-	-
Просмотр заказов	-	✔ (свои)	✔ (свои)	✔ (ограниченно)	✔ (агрегированно)	✔
Управление заказами	-	-	✔ (свои)	✔ (споры)	-	✔
Платежи	-	✔	-	-	-	-
Финансовые операции	-	-	✔ (свои)	✔ (ограниченно)	✔ (просмотр)	✔
Доступ к персональным данным	-	свои	по заказам	ограниченно	агрегированно	✔

Операция / Роль	Гость	Покупатель	Продавец	Модератор	Аналитик	Администратор
Управление доступом	-	-	-	-	-	✔

Атрибутный контроль доступа (ABAC)

RBAC дополняется атрибутами для критичных операций.

Атрибуты

- пользователь
- роль
- владелец ресурса
- признак прохождения 2FA
- тип операции
- чувствительность данных
- контекст доступа (IP, география, сессия)

Правила

- Доступ к ресурсам:
- только владелец или привилегированная роль
- Заказы:
- доступ только участникам
- Финансовые операции:
- обязательная 2FA
- проверка владения
- Персональные данные:
- ограничение по чувствительности
- Административные операции:
- повышенные требования (роль + 2FA + ограниченный доступ)

4.1.3. Аутентификация и сессии

- OAuth2 / OpenID Connect (при необходимости)
- Локальная аутентификация (логин/пароль + 2FA)
- Сессии:

- завершение при неактивности (30 минут)
- refresh-токены с ограниченным сроком
- принудительное завершение

Двухфакторная аутентификация

Обязательна для:

- финансовых операций
- изменения чувствительных данных
- административных действий

Методы:

- OTP (SMS / push)
- TOTP

4.1.4. Управление секретами и ключами

Типы секретов

- JWT-ключи
- пароли пользователей
- API-ключи интеграций
- ключи шифрования данных
- TLS-сертификаты

Хранение

- централизованная система управления секретами
- ограниченный доступ по ролям
- аудит операций

Жизненный цикл

- генерация централизованно
- регулярная ротация
- отзыв при компрометации

4.1.5. Шифрование

- Данные в БД — AES-256 (на уровне приложения, sensitive fields)
- Передача — TLS 1.2+
- Межсервисное взаимодействие — защищённые соединения (mTLS)

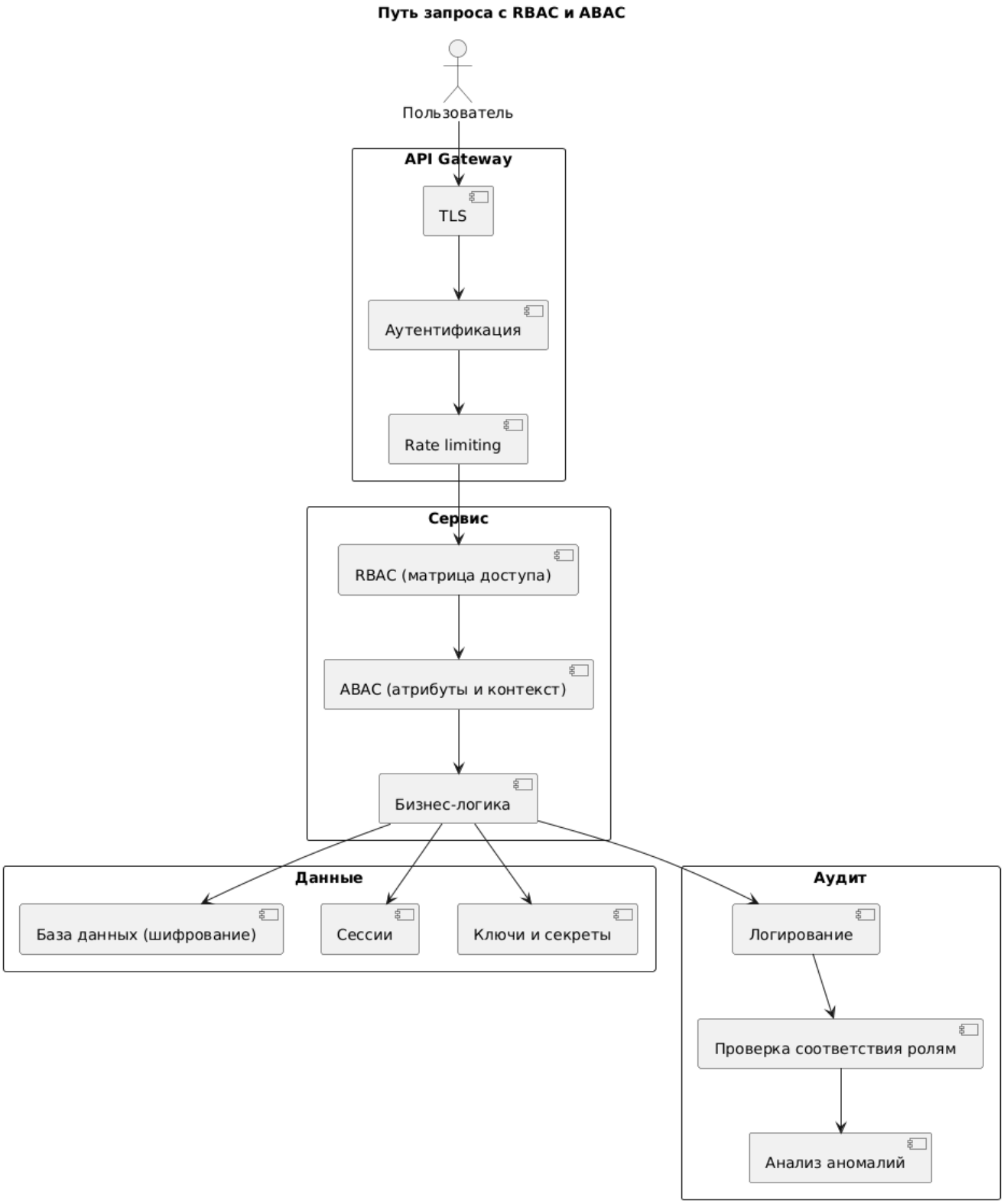
4.1.6. Реализация контроля доступа

- API Gateway:
- аутентификация
- rate limiting
- Сервисы:
- RBAC
- ABAC
- JWT:
- содержит роль и базовые атрибуты

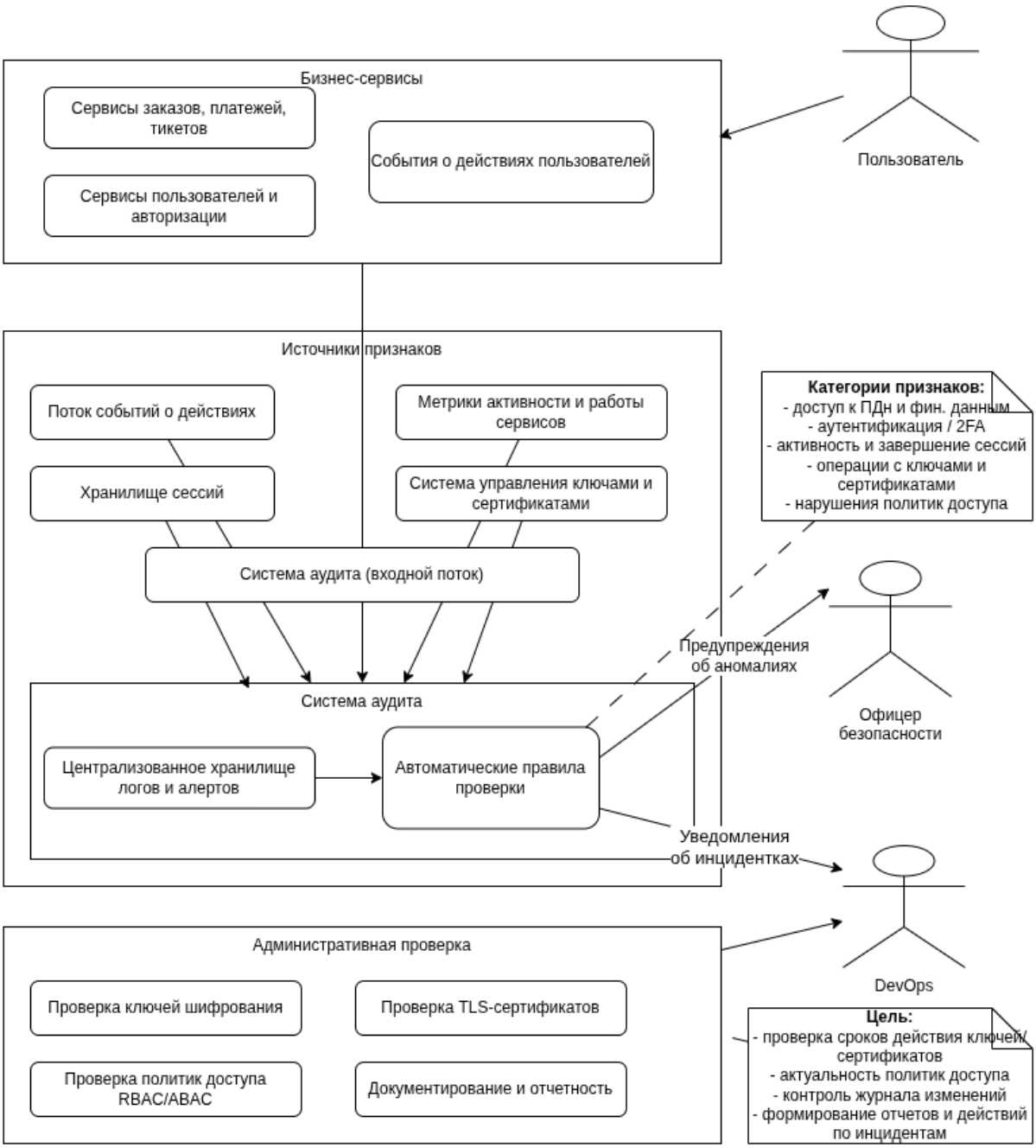
4.1.7. Защита от атак

- DDoS — через CDN и внешние фильтры
- Rate limiting - API Gateway
- Защита от SQL/XSS - сервисы на PostgreSQL (SQL); Frontend, API Gateway, сервисы, возвращающие HTML/контент (XSS)
- Валидация входных данных

4.1.8. Путь запроса с проверкой полномочий



4.2. Аудит безопасности



4.2.1. Цели

- Контроль доступа к персональным и финансовым данным.
- Выявление попыток несанкционированного доступа и аномалий.
- Обеспечение соответствия требованиям 152-ФЗ и корпоративным стандартам.

- Поддержка расследований инцидентов.

4.2.2. Объекты аудита

- Доступ к персональным данным и финансовым операциям.
- Изменения ролей пользователей и политик доступа.
- Аутентификация и операции с токенами.
- Двухфакторная аутентификация для критичных действий.
- Операции с ключами шифрования и TLS-сертификатами.
- Состояние пользовательских сессий.
- Все критичные бизнес-операции: тикеты, заказы, платежи, сообщения.

4.2.3. Источники данных

- Поток событий бизнес-сервисов (неизменяемые записи о действиях).
- Централизованное хранилище логов с поиском и алертами.
- Метрики работы сервисов и активности пользователей.
- Хранилище сессий для контроля завершения и времени последней активности.
- Система управления ключами и сертификатами.
- Системные журналы ОС и платформы оркестрации контейнеров.

4.2.4. Роли

- **Пользователь** — источник событий аудита.
- **DevOps** — обеспечение корректной работы инфраструктуры аудита:
- сбор и хранение логов и метрик
- поддержка KMS и TLS
- контроль целостности хранилищ сессий
- обеспечение работоспособности автоматических правил проверки
- **Офицер безопасности** — анализ событий, выявление аномалий, административная проверка ключей, сертификатов и политик, расследование инцидентов, формирование официальных отчетов.

4.2.5. Методы контроля

- Автоматическая проверка аномалий по событиям и метрикам.
- Контроль выполнения политик доступа, двухфакторной аутентификации и времени жизни сессий.
- Мониторинг операций с ключами и сертификатами.
- Формирование алертов и уведомлений офицерам безопасности.

4.2.6. Управление инцидентами

- Критичные события вызывают автоматические уведомления.
- При подозрении на компрометацию ключей, токенов или персональных данных:
  - Ротация ключей и сертификатов
  - Завершение активных сессий
  - Форензик и расследование через логи и метрики
  - Ведение журнала инцидентов и отчётность.

4.2.7. Административная проверка

- Ручной или полуавтоматический аудит ключей, сертификатов и политик.
- **Проверка ключей шифрования:** срок действия, актуальность версий, доступ, логирование операций.
- **Проверка TLS-сертификатов:** срок действия, подпись доверенным CA, корректное использование, отсутствие дублирующих сертификатов.
- **Проверка политик доступа RBAC/ABAC:** соответствие стандартам, актуальность, контроль критичных операций, журнал изменений.
- **Документирование и отчетность:** кто проверял, когда, результаты, меры, хранение отчетов для внутреннего аудита и соответствия 152-ФЗ.
- **Частота:** ежемесячно для ключей, сертификатов и политик; дополнительная проверка при значительных изменениях.

4.2.8. Контроль соответствия ролям

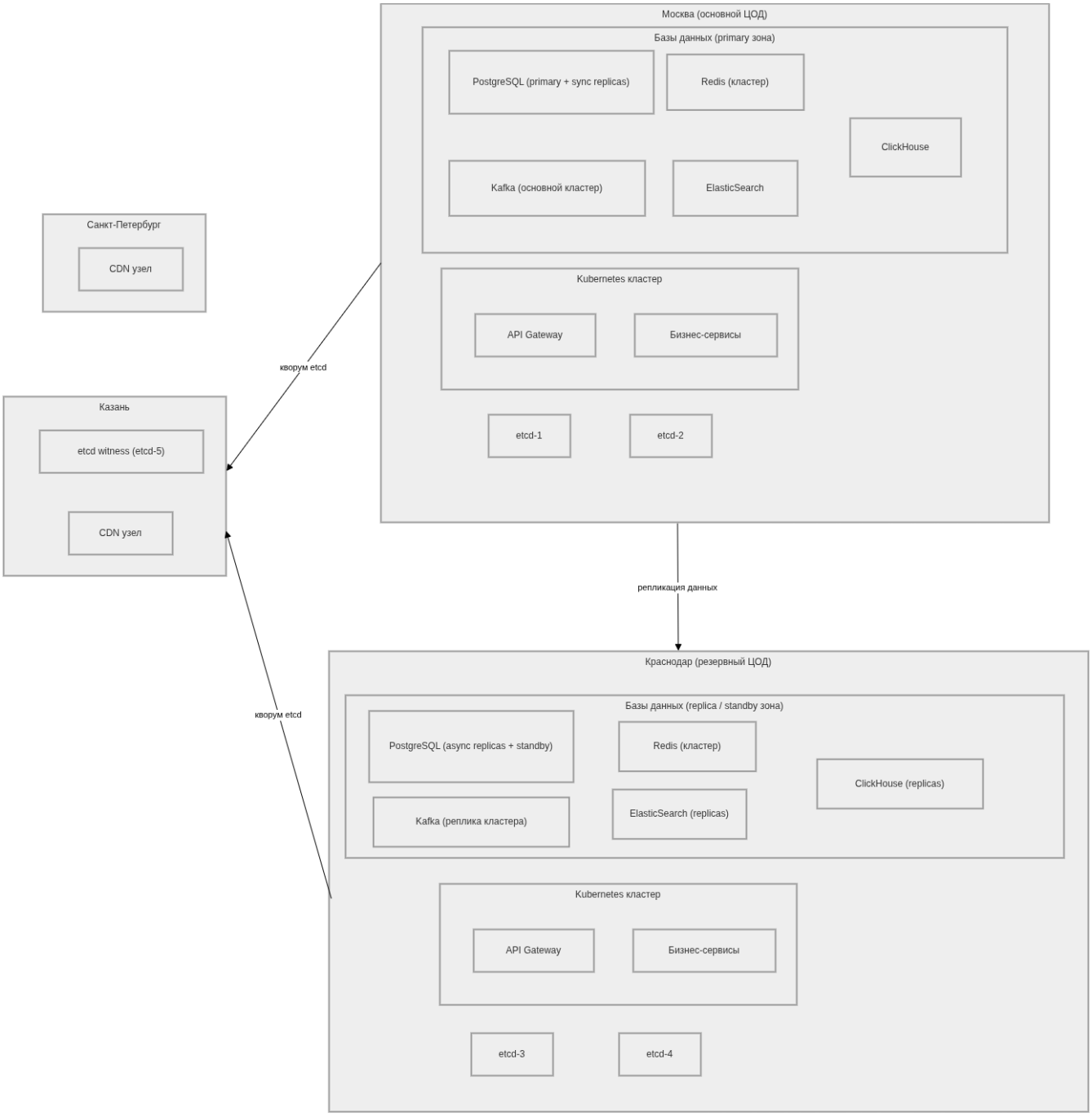
- Проверка выполняется на основе матрицы доступа (RBAC).
- Каждое действие пользователя сопоставляется допустимым операциям его роли.

Примеры нарушений:

- выполнение операции вне разрешенной матрицы
- доступ к чужим ресурсам

Глава 5. Доставка решения

5.1. Схема развертывания



5.1.1. География и ЦОД

- Москва — северный регион, основной ЦОД (primary для критичных записей)
- Краснодар — южный регион, secondary / standby (частично active-active для некритичных сервисов)
- Казань — witness-узел (узел кворума для отказоустойчивости)
- Между ЦОД:
  - асинхронная репликация данных (по умолчанию)

    - синхронная репликация внутри ЦОД для критичных сервисов
- CDN для медиа:
    - edge: СПб и Казань

5.1.2. Сервисы и распределение по ЦОД

API и бизнес-сервисы

- API Gateway — контейнеры
    - Москва: 4 реплики
    - Краснодар: 2 реплики
    - Stateless, active-active

- Сервис авторизации — контейнеры + базы (VM/bare-metal)
    - PostgreSQL (users, sessions)
        - Москва: Master shard 1
        - Краснодар: Master shard 2
        - Реплики shard 1 в Краснодар, shard 2 в Москву
        - Локальная запись (по географии)
- Redis cluster (sessions + tokens)
    - Москва: 3 master + 3 replica
    - Краснодар: 3 master + 3 replica
    - Cluster mode, hash-slot
    - Все записи пользователей и сессий идут в локальный мастер ЦОД
    - Асинхронная меж-ЦОД репликация

- Сервис пользователей — контейнеры + PostgreSQL (VM/bare-metal)
    - Москва: Master shard 1
    - Краснодар: Master shard 2
    - Реплики shard 1 в Краснодар, shard 2 в Москву
    - Обеспечивает локальную запись пользователей и отказоустойчивость при падении одного ЦОД

- Сервис объявлений — контейнеры + PostgreSQL (VM/bare-metal)

    - Москва: Master shards 1,2; Replica shard 3
    - Краснодар: Master shard 3; Replica shards 1,2
    - Geo-sharding: запись в локальный мастер, реплика в другой ЦОД
    - Шардирование обеспечивает балансировку нагрузки и отказоустойчивость

- Сервис поиска + рекомендации — контейнеры + ElasticSearch (VM/bare-metal)
    - Москва: 3 data node
    - Краснодар: 3 data node
    - replication factor = 1 (меж-ЦОД)
    - eventual consistency

- Сервис заказов, платежей, расчётов — контейнеры + PostgreSQL (VM/bare-metal)
    - Модель: Single-writer (единственный primary)
    - Развёртывание:
        - Москва:
            - - 1 primary
            - - 1 synchronous replica
        - Краснодар:
            - - 1 asynchronous replica (standby)
    - Failover:
        - - автоматический, при кворум (обеспецивается etcd, см. ниже)
        - - при недоступности primary временная остановка записи до принятия решения

- Сервис доставки — контейнеры + PostgreSQL (VM/bare-metal) (критичный контур)
    - Geo-sharding по региону
    - Master в локальном ЦОД, replica в другом

- Аналитический сервис — контейнеры + ClickHouse (VM/bare-metal)
    - Москва: 2 ноды
    - Краснодар: 2 ноды
    - Асинхронная репликация

- Сервис медиа — S3 + CDN

    - S3 Москва (primary)
    - S3 Краснодар (replica)
    - CDN edge: СПб, Казань

Технические компоненты

- Redis cluster (cache)
    - Москва: 3 master + 3 replica
    - Краснодар: 3 master + 3 replica
    - Асинхронная репликация

- Kafka — event bus
    - Москва: 3 брокера
    - Краснодар: 3 брокера
    - replication factor = 3
    - Меж-ЦОД распределение партиций

- etcd cluster (кворум при падениях)
    - Назначение:
        - определение "жив ли primary", предотвращение двух primary для критичных сервисов
        - защита от split-brain
        - координация failover PostgreSQL (через Patroni)
    - Состав:
        - Москва: 2 ноды
        - Краснодар: 2 ноды
        - Казань: 1 witness node (резервные ноды, кворум = 3/5)

- Мониторинг — ELK stack
    - Москва: 1 кластер
    - Краснодар: 1 кластер
    - Централизованное логирование

```
Метрики и health-checks — Prometheus + Grafana
```

- Развёртывание:
  - Москва: 2 инстанса Prometheus
  - Краснодар: 2 инстанса Prometheus
- Дополнительно:
  - health agents
    - проверка:
      - доступности PostgreSQL primary
      - доступности ЦОД (network checks)
    - Использование:
      - сигнализация отказов
      - триггеры для failover (через интеграцию с etcd/Patroni)

5.1.3. Принципы

- Stateless-сервисы:
  - Kubernetes, active-active
- Stateful-сервисы (БД etc.):
  - локальный primary + реплика в другом ЦОД
- Критичные сервисы (Payments, Orders):
  - single-writer модель
  - failover только через кворум etcd
  - при неопределённости запись останавливается
- Geo-sharding:
  - локальная запись для пользователей и объявлений (Ticket)
- Failover:
  - автоматический, но с проверкой quorum
  - Kubernetes - пересоздание контейнеров
  - PostgreSQL - переключение на реплику только при подтверждении etcd
  - GeoDNS - переключение трафика

5.2. CI/CD процесс

5.2.1. Инструменты

- Git: GitVerse
- CI/CD: GitVerse CI/CD
- Контейнеризация: Docker
- Оркестрация: Kubernetes
- Infrastructure as Code: OpenTofu
- Упаковка приложений: Helm
- Registry: GitVerse Container Registry
- Мониторинг и alerting: Prometheus + Grafana
- Логирование: ELK Stack
- Миграции баз данных: Liquibase, Flyway
- Секреты: Vault

5.2.2. Уровни критичности сервисов

Сервисы разделены на три типа. Тип определяет стратегию деплоя, жёсткость порогов rollback и требования к подтверждению.

Тип	Сервисы	Характеристика
1 — Финансовые	Платёжный сервис, Расчёты, Заказы	Нулевая толерантность к ошибкам. Потеря транзакций недопустима.
2 — Ядро	Авторизация, Пользователи, Объявления, Доставка	Высокая доступность. Деградация функциональности недопустима.
3 — Операционные	Поиск, Медиа, Аналитика, Нотификации	Допустима кратковременная деградация без потери данных.

5.2.3. Архитектура пайплайнов

Система использует **три отдельных пайплайна**, разделённых по назначению и чувствительности данных.

Пайплайн бизнес-сервисов

- Сервисы: API Gateway, Авторизация, Пользователи, Объявления, Поиск, Заказы, Платёжный сервис, Расчёты, Доставка, Аналитика/Нотификации
- Контейнеризация: Docker + Kubernetes
- Стратегия деплоя определяется типом сервиса

Сервис	Тип	Стратегия деплоя
Платёжный сервис	1	Blue/Green + ручное подтверждение
Расчёты	1	Blue/Green + ручное подтверждение
Заказы	1	Blue/Green + ручное подтверждение
Авторизация	2	Canary
Пользователи	2	Canary
Объявления	2	Canary
Доставка	2	Canary
API Gateway	2	Canary
Поиск	3	Rolling update
Медиа	3	Rolling update
Аналитика / Нотификации	3	Rolling update

Пайплайн баз данных и инфраструктуры

- Сервисы: PostgreSQL, Redis, Kafka, ClickHouse, Elasticsearch
- Развёртывание: VM / bare-metal
- Стратегия: управление конфигурацией через OpenTofu, репликация между ЦОД, автоматическое тестирование восстановления
- Миграции БД: только через Flyway; обязательный бэкап перед применением; ручное подтверждение для Типа 1 и 2

Пайплайн медиа и CDN

- Сервисы: S3/Object Storage, CDN edge (СПб, Казань)
- Развёртывание: managed
- Стратегия: автоматическое обновление правил кэширования и деплой новых версий контейнеров медиа-сервисов; инвалидация CDN-кэша выполняется последней — после успешного завершения деплоя в обоих ЦОД

5.2.4. Этапы CI/CD

CI

Trigger: push / merge request в GitVerse.

1. Сборка Docker-образа
2. Unit-тесты (покрытие > 80%)
3. Интеграционные тесты взаимодействия сервисов

4. Статический анализ: SonarQube, включая проверки на SQL-инъекции и XSS
5. Security scan: Trivy, Grype (CVE-анализ зависимостей и образов)
6. Сохранение артефактов в GitVerse Container Registry

CD

Окружения

- **Development** — автоматический деплой feature-веток
- **Preproduction** — полная копия production; интеграционное и e2e тестирование
- **Production** — деплой по расписанию, ночное время 02:00–04:00

Стратегия деплоя по типам

Тип	Стратегия	Особенности
1	Blue/Green + ручное подтверждение	Pipeline останавливается перед переключением трафика. Переход подтверждается инженером. Старая версия (blue) остаётся активной 30 минут после переключения. Rollback = переключение балансировщика, ~30 сек. Для обеспечения репрезентативной выборки при низком ночном трафике применяется replay обезличенных production-транзакций.
2	Canary	5% трафика → наблюдение 10 мин → 25% → наблюдение 10 мин → 100%. Автоматический rollback по метрикам.
3	Rolling update	Замена по одной реплике. Автоматический rollback по метрикам.

Порядок деплоя по ЦОД

Деплой всегда выполняется в фиксированном порядке во избежание одновременного отказа обоих ЦОД.

1. Москва (primary ЦОД) — наблюдение N запросов + 15 мин
2. Краснодар — наблюдение N запросов + 15 мин
3. Инвалидация CDN-кэша (СПб, Казань)

При неуспешном завершении шага 1 шаги 2 и 3 не запускаются автоматически.

Полный pipeline

1. **Pre-deploy checks** (блокирующие):
    - etcd-кворум достигнут

    - Лаг репликации БД < 30 сек
    - Kafka consumer lag в норме (нет активного накопления)
    - Текущий error rate не превышает baseline (отсутствие активного инцидента)
2. Создание Helm-чарта с новой версией
3. Развёртывание в preproduction
4. Запуск интеграционных и e2e тестов (обязательные сценарии: оплата, размещение объявления, доставка)
5. Деплой в production — Москва, в соответствии со стратегией типа
6. Наблюдение: накопление минимальной выборки N, затем 15 мин мониторинга
7. Деплой в production — Краснодар
8. Наблюдение: накопление минимальной выборки N, затем 15 мин мониторинга
9. Инвалидация CDN-кэша (СПб, Казань)
10. Финальная верификация 30 мин; снятие нового baseline

5.2.5. Мониторинг после деплоя и rollback

Принцип задания порогов

Деплой в production происходит в период минимальной нагрузки (~02:00–04:00), когда абсолютные значения метрик нетипичны для эксплуатационных условий. Поэтому пороги rollback задаются не абсолютными значениями, а как **коэффициент отклонения от исторического baseline** того же временного окна (±30 минут, скользящее среднее за последние 4 недели).

```
Rollback если:
  error_rate(now)   > baseline_same_window x K_err
  p95_latency(now)  > baseline_same_window x K_lat
  p99_latency(now)  > baseline_same_window x K_p99
```

Оценка метрик не начинается до накопления минимальной выборки N запросов (защита от ложных срабатываний при первых секундах после деплоя).

Пороги по типам

Метрика	Тип 1	Тип 2	Тип 3	Применение
Минимальная выборка N	50 req	100 req	200 req	Старт оценки
K error rate	×1.5	×2.0	×2.5	Относительно baseline
K p95 latency	×1.3	×1.5	×2.0	Относительно baseline

Метрика	Тип 1	Тип 2	Тип 3	Применение
K p99 latency	×1.2	×1.4	—	Относительно baseline

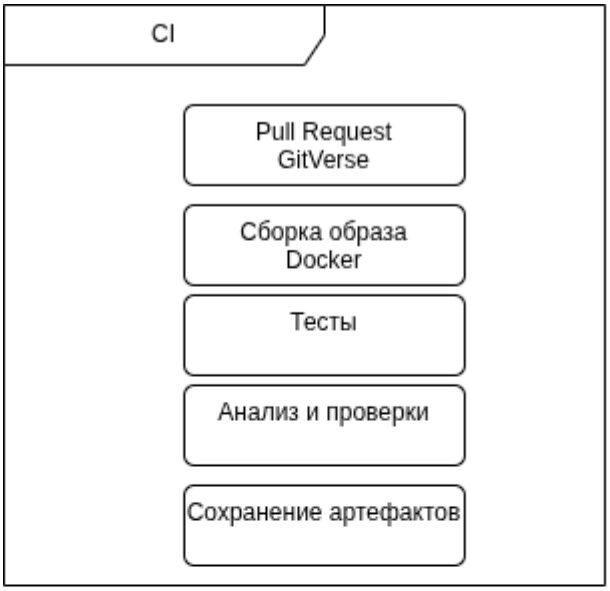
Дополнительные блокирующие метрики (все типы)

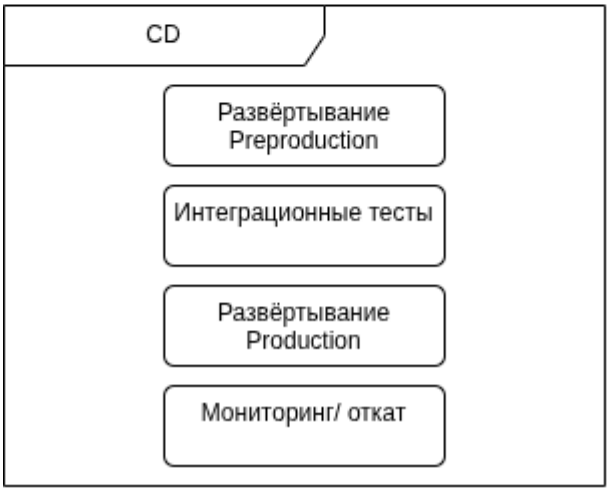
Метрика	Порог	Действие
Kafka consumer lag	> 50 000 сообщений	Rollback
Лаг репликации БД	> 30 сек	Остановить деплой, не начинать следующий ЦОД
Redis hit rate	падение > 20% от baseline	Alert, ручное решение
Elasticsearch index lag	> 60 сек	Rollback (Тип 3)
etcd quorum	не достигнут	Запретить деплой

Конкретные значения коэффициентов K и минимального объёма выборки N уточняются в runbook каждого сервиса по результатам нагрузочного тестирования и накопленной статистики baseline.

Уведомления и логирование

- Health checks всех сервисов после каждого этапа деплоя
- Метрики в Prometheus/Grafana; дашборд деплоя отдельным стендом
- Автоматические уведомления администраторов при rollback и критических событиях
- Централизованное логирование через ELK; все операции с финансовыми данными логируются отдельно с меткой `audit`





5.2.6. Планы тестирования

Назначение

Цель тестирования — подтвердить:

- корректную работу в распределённой multi-DC архитектуре
- поддержку средних и пиковых нагрузок
- устойчивость событийной обработки (Kafka + ClickHouse)
- корректность разделения OLTP (PostgreSQL) и OLAP (ClickHouse)
- отсутствие потерь данных (включая события)
- соблюдение SLA по задержкам и восстановлению

5.2.7. Объекты тестирования

Точки входа

- GeoDNS
- API Gateway
- CDN

Бизнес-сервисы

- User / Auth
- Ticket (объявления + сообщения)
- Order
- Payment / Billing
- Delivery
- Search / Recommendation
- Analytics (включая нотификации)

Инфраструктурные компоненты

- PostgreSQL
- Redis
- Kafka
- Elasticsearch
- ClickHouse
- S3 + CDN
- ELK / Prometheus / Grafana

5.2.8. Архитектурные особенности

Учитываются:

- 2 активных ЦОД (Москва, Краснодар)
- eventual consistency между регионами
- событийная архитектура (Kafka)
- разделение OLTP и OLAP
- задержки обработки событий (от генерации до аналитики)
- повторная обработка событий как механизм восстановления

5.2.9. Основные группы тестирования

Функциональное тестирование

Проверяется end-to-end:

- регистрация / логин
- создание объявления
- поиск и выдача
- оформление заказа
- оплата
- доставка

Дополнительно:

- корректность формирования событий в Kafka
- попадание событий в ClickHouse
- генерация нотификаций аналитическим сервисом

Проверка производительности

Проверяются задержки обработки запросов и пропускная способность системы.

Сценарии:

- загрузка страницы каталога
- выполнение поискового запроса
- создание заказа
- инициация платежа

Метрики (под нагрузкой):

- API latency:
  - ≤ 2 сек (P95)
  - ≤ 4 сек (P99)
- search latency:
  - ≤ 5 сек (P95)
- payment initiation:
  - ≤ 5 сек (P95)

Дополнительно:

- concurrent users:

    - ≥ 10 000 (целевое)
    - ≥ 15 000 (пиковое)

Проверка согласованности данных

Проверяется:

- PostgreSQL — источник истины
- Elasticsearch — синхронизация индекса ≤ 2 мин
- Redis — согласованность TTL ≤ 60 сек
- ClickHouse:
    - задержка событий ≤ 60–180 сек
    - отсутствие потерь событий (через повторную обработку)
- допустимая eventual consistency:
    - расхождение OLTP и OLAP ≤ 5 минут

Проверка событийной архитектуры

Проверяется:

- идемпотентность обработки событий
- обработка дублирующихся событий
- обработка событий вне порядка
- повторная обработка событий (replay)

Дополнительно:

- доставка событий через Kafka:
    - success rate ≥ 99.99%
- корректность работы групп потребителей при масштабировании
- восстановление аналитики через повторное чтение событий (Kafka → ClickHouse)

Проверка распределённой системы (2 ЦОД)

Проверяется:

- переключение GeoDNS ≤ 60 сек
- отказ одного ЦОД:
    - деградация по задержкам ≤ 30%

- восстановление:
  - ≤ 15 минут частичное восстановление
- отсутствие потерь заказов и платежей

Проверка устойчивости к сбоям

Сценарии:

- отказ shard PostgreSQL
- отказ узла Redis
- остановка брокера Kafka
- деградация узла ClickHouse
- отказ одного ЦОД

Критерии:

- система продолжает работу с деградацией
- восстановление сервисов ≤ 15 минут
- восстановление данных через повторную обработку ≤ 6 часов

Проверка восстановления

Проверяется:

- восстановление PostgreSQL из реплики
- восстановление аналитики через повторную обработку Kafka → ClickHouse
- восстановление индекса Elasticsearch
- восстановление Redis cache

Проверка безопасности

- TLS 1.2+
- шифрование данных (AES-256)
- rate limiting:
  - ≤ 100 req/sec на пользователя (по умолчанию)
- защита от SQL/XSS
- проверка подписи webhook

5.2.10. Критические зоны системы

- платежи
- заказы
- объявления
- поиск
- авторизация
- событийный конвейер (Kafka → ClickHouse)

5.2.11. Общий критерий успешности тестирования

Система считается корректной, если:

- 100% критических сценариев проходят успешно
- потери транзакций: 0
- потери событий: 0 (с учетом повторной обработки)
- соблюдение SLA:
  - latency ≤ 2 сек (P95)
  - search ≤ 5 сек (P95)
- система выдерживает:
  - ≥ 10 000 concurrent users (стабильно)
  - ≥ 100 000 событий/сек
- восстановление:
  - сервисы ≤ 15 мин
  - данные ≤ 1 час (OLTP), ≤ 6 часов (OLAP)
- георасределение не приводит к превышению допустимого уровня рассогласования